MODULE 3

Clipping and Projections

Window to viewport transformation. Cohen Sutherland Line clipping algorithm. Sutherland Hodgeman Polygon clipping algorithm. Three-dimensional viewing pipeline. Projections- Parallel and Perspective projections. Visible surface detection algorithms- Depth buffer algorithm, Scan line algorithm.

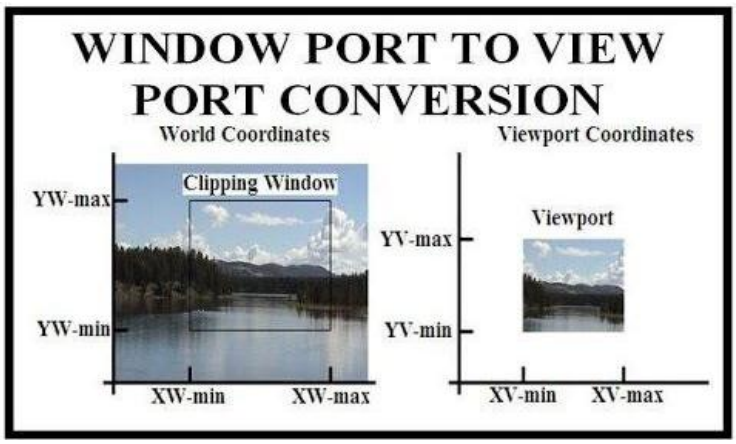
WINDOWING CONCEPTS – TWO-DIMENSIONAL CLIPPING.

The primary use of clipping in computer graphics is to remove objects, lines, or line segments that are outside the viewing pane. The viewing transformation is insensitive to the position of points relative to the viewing volume − especially those points behind the viewer − and it is necessary to remove these points before generating the view.

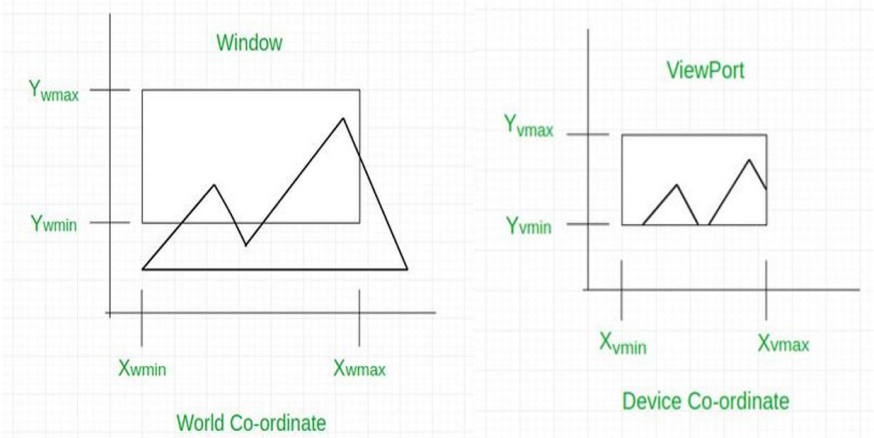
Window port and View port

A window defines a rectangular area in world coordinates. You can define the window to be larger than, the same size as, or smaller than the actual range of data values, depending on whether you want to show all of the data or only part of the data.

A viewport defines in normalized coordinates a rectangular area on the display device where the image of the data appears.



A windowing system is a system for sharing a computer's graphical display presentation resources among multiple applications at the same time. In a computer that has a graphical user interface (GUI), you may want to use several applications at the same time (this is called task).



- **World coordinate** – It is the Cartesian coordinate w.r.t which we define the diagram, like Xwmin, Xwmax, Ywmin, Ywmax
- **Device Coordinate** –It is the screen coordinate where the objects are to be displayed, like Xvmin, Xvmax, Yvmin, Yvmax
- **Window** –It is the area on world coordinate selected for display.
- **View Port** –It is the area on the device coordinate where graphics is to be displayed.

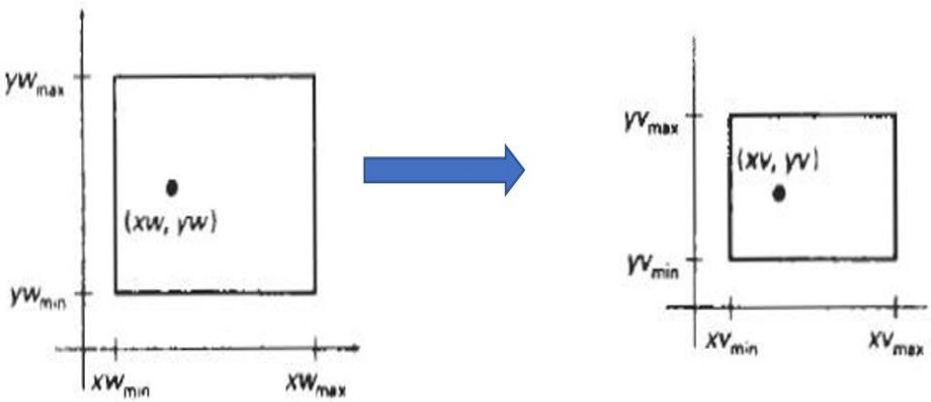
It may be possible that the size of the Viewport is much smaller or greater than the Window.

we must increase or decrease the size of the Window according to the Viewport and for this, we need some mathematical calculations.

(xw, yw): A point on Window

(xv, yv): Corresponding point on Viewport

we must calculate the point (xv, yv)



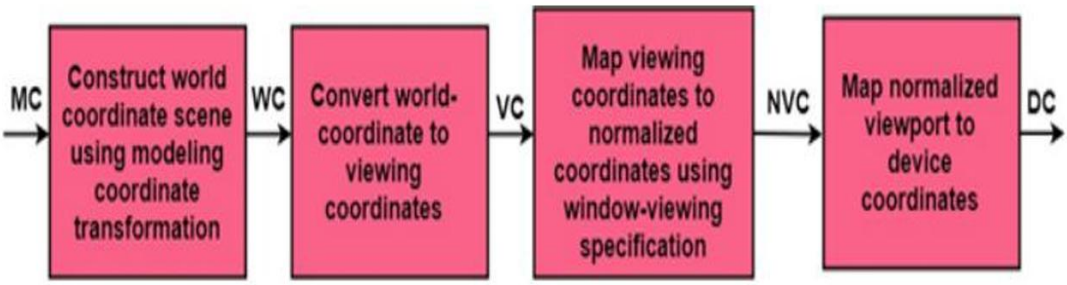
<u>2D Viewing transformation pipeline</u>

• Construct the scene in world coordinate using the output primitives and attributes.

• To obtain a particular orientation, we can set up a 2-D viewing coordinate system in the window coordinate plane and define a window in viewing coordinates system.

• Once the viewing frame is established, then transform description in world coordinates to viewing coordinates.
• Define viewport in normalized coordinates (range from 0 to 1) and map the viewing coordinates description of the scene to normalized coordinates.
• All parts of the picture that (i.e., outside the viewport are dipped, and the contents are transferred to device coordinates).

2D Viewing transformation pipeline



- **By changing the position of the viewport:** We can view objects at different locations on the display area of an output device as shown in fig:
-

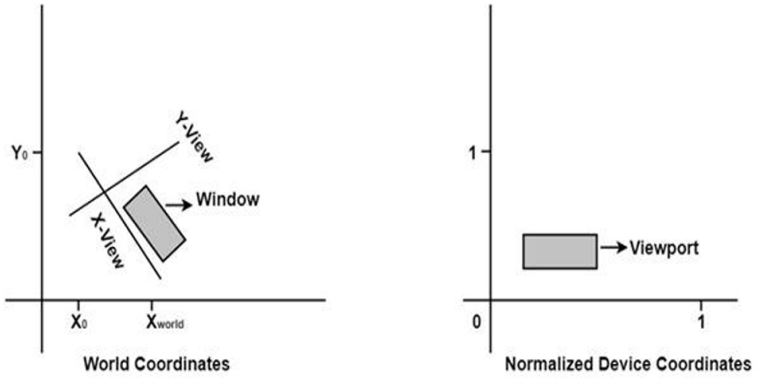


Fig:Setting up a rotated world window and corresponding normalized coordinate viewport.

- By varying the size of viewports:
- We can change the size and proportions of displayed objects.
- We can achieve zooming effects by successively mapping different- sized windows on a fixed-size viewport.

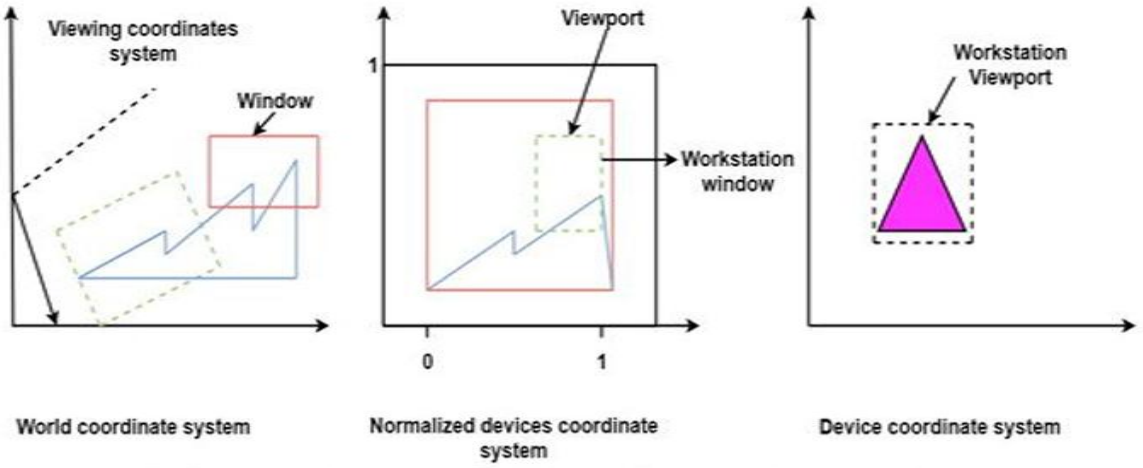
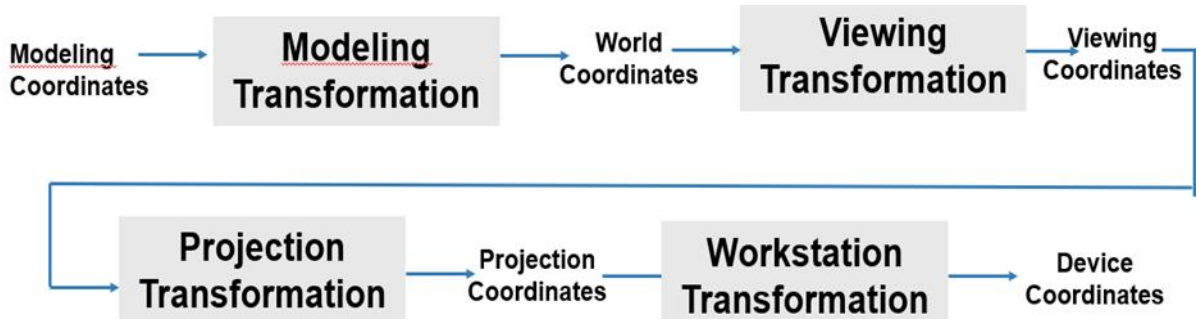


Fig: Zooming effects by mapping different-sized windows on a fixed-size viewport.

Describe about 3D viewing pipeline

The steps for computer generation of a view of a three-dimensional scene are somewhat analogous to the processes involved in taking a photograph.



WINDOW COORDINATE → NORMALISED COORDINATE → DEVICE COORDINATES (within (0,1))

- Window coordinate to normalized coordinate translation.
- Scaling in the normalised coordinate system in order to make it match with viewport (device coordinate system)
- Translate to Device coordinate (view port) system.
- To maintain the same relative placement of the point in the viewport as in the window, we require:

$$\text{Normalized Point on Window} \quad \left(\frac{X_w - X_{wmin}}{X_{wmax} - X_{wmin}}, \frac{Y_w - Y_{wmin}}{Y_{wmax} - Y_{wmin}} \right)$$

$$\text{Normalized Point on Viewport} \quad \left(\frac{X_v - X_{vmin}}{X_{vmax} - X_{vmin}}, \frac{Y_v - Y_{vmin}}{Y_{vmax} - Y_{vmin}} \right)$$

Now the relative position of the object in Window and Viewport are same

$$\frac{X_w - X_{wmin}}{X_{wmax} - X_{wmin}} = \frac{X_v - X_{vmin}}{X_{vmax} - X_{vmin}}$$

$$Xv = \frac{(Xw - Xwmin)(Xvmax - Xvmin)}{(Xwmax - Xwmin)} + Xvmin$$

$$X_V = X_{vmin} + (X_w - X_{wmin})\, S_x$$

where s_x is the scaling factor of x coordinate and s_y is the scaling factor of y coordinate

$$\frac{Y_w - Y_{wmin}}{Y_{wmax} - Y_{wmin}} = \frac{Y_v - Y_{vmin}}{Y_{vmax} - Y_{vmin}}$$

$$Yv = \frac{(Yw - Ywmin)(Yvmax - Yvmin)}{(Ywmax - Y\,wmin)} + Yvmin$$

$$Y_V = Y_{vmin} + (Y_w - Y_{wmin})\, Sy$$

$$S_x = \frac{X_{vmax} - X_{vmin}}{X_{wmax} - X_{wmin}} \qquad S_y = \frac{Y_{vmax} - Y_{vmin}}{Y_{wmax} - Y_{wmin}}$$

Problems:-

1 Q) for window, Xwmin = 20, Xwmax = 80, Ywmin = 40, Ywmax = 80.

for viewport, Xvmin = 30, Xvmax = 60, Yvmin = 40, Yvmax = 60.

Now a point (Xw, Yw) be (30, 80) on the window. We must calculate that point on the viewport i.e (Xv, Yv).

$$S_x = \frac{X_{vmax} - X_{vmin}}{X_{wmax} - X_{wmin}} \qquad S_y = \frac{Y_{vmax} - Y_{vmin}}{Y_{wmax} - Y_{wmin}}$$

Sx = (60 - 30) / (80 - 20) = 30 / 60
Sy = (60 - 40) / (80 - 40) = 20 / 40

So, now calculate the point on the viewport (Xv, Yv).

$$X_V = X_{vmin} + (X_W - X_{wmin})\, S_x$$

Xv = 30 + (30 - 20) * (30 / 60) = 35

$$Y_V = Y_{vmin} + (Y_W - Y_{wmin})\, S_y$$

Yv = 40 + (80 - 40) * (20 / 40) = 60

So, the point on window (Xw, Yw) = (30, 80) will be (Xv, Yv) = (35, 60) on viewport.

Q2) For a window with coordinates,
Xwmin = 20, Xwmax = 80, Ywmin = 40, Ywmax = 80.
And viewport coordinates,
Xvmin = 30, Xvmax = 60, Yvmin = 40, Yvmax = 60.
A point (Xw, Yw) be (30, 80) on the window.
Calculate that point on the viewport i.e (Xv, Yv).

Solution:-

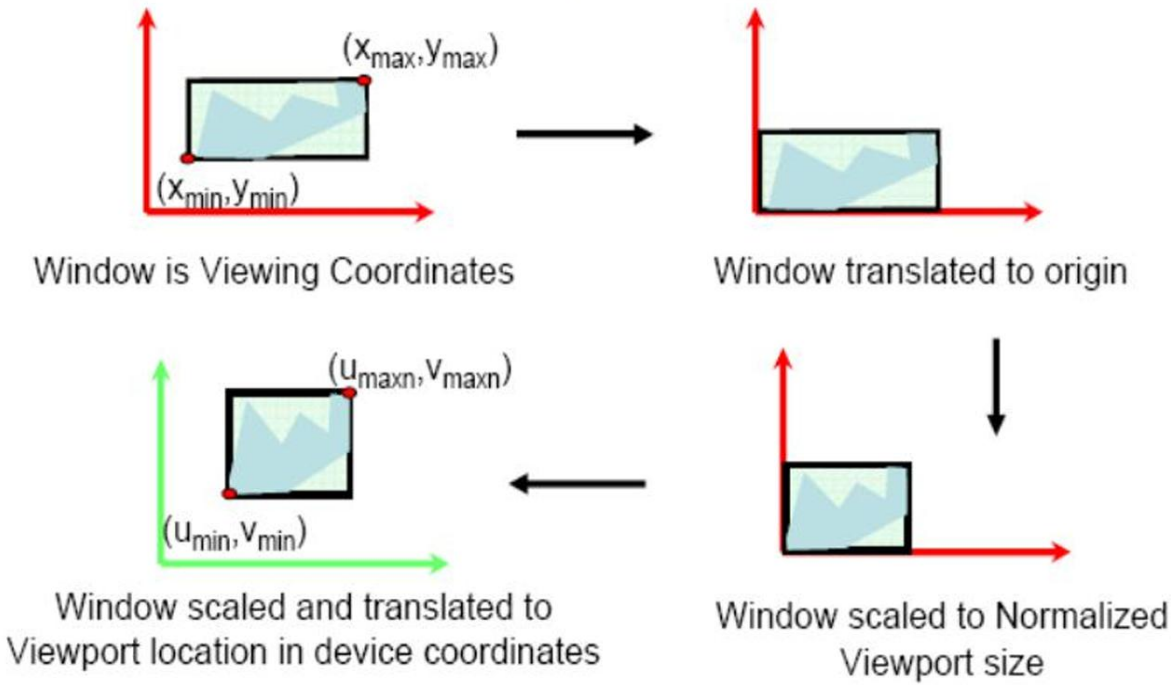
$$S_x = \frac{X_{vmax} - X_{vmin}}{X_{wmax} - X_{wmin}} \qquad S_y = \frac{Y_{vmax} - Y_{vmin}}{Y_{wmax} - Y_{wmin}}$$

Sx = (60 - 30) / (80 - 20) = 30 / 60
Sy = (60 - 40) / (80 - 40) = 20 / 40

$$X_V = X_{vmin} + (X_w - X_{wmin})\, S_x \qquad Y_V = Y_{vmin} + (Y_w - Y_{wmin})\, S_y$$

- So, now calculate the point on the viewport (X_v, Y_v).

$X_v = 30 + (30 - 20) * (30 / 60) = 35$
$Y_v = 40 + (80 - 40) * (20 / 40) = 60$
So, the point on window (X_w, Y_w) = (30, 80) will be **(X_v, Y_v) = (35, 60)** on viewport.



Steps for window to viewport transformation

- Step 1: Translate window towards origin

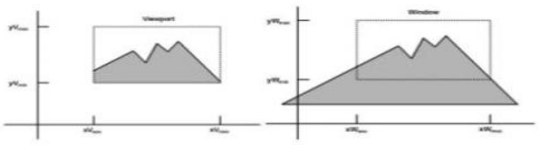
To shift window towards origin, lower left or upper left corner of window will become (-).
Hence translation factor will become negative (-tx,-ty).

- Step 2: Resize window to the size of view port.

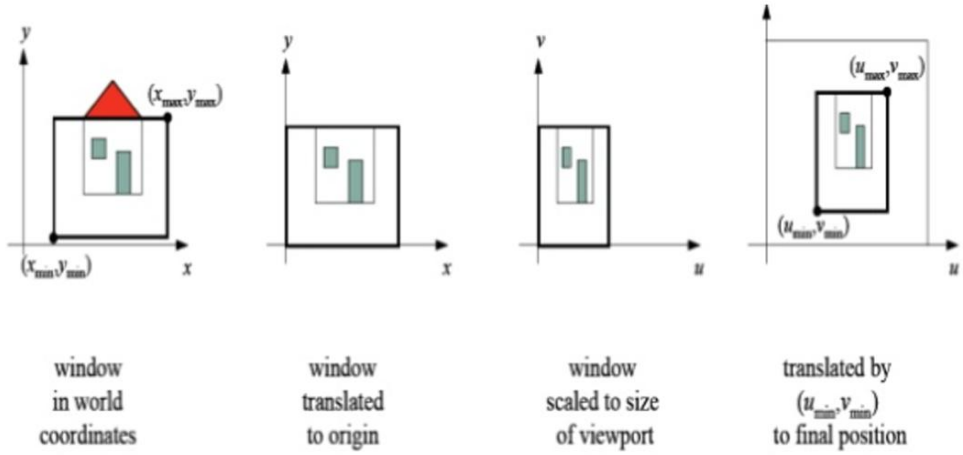
To convert window size in to view port size following computation is required.

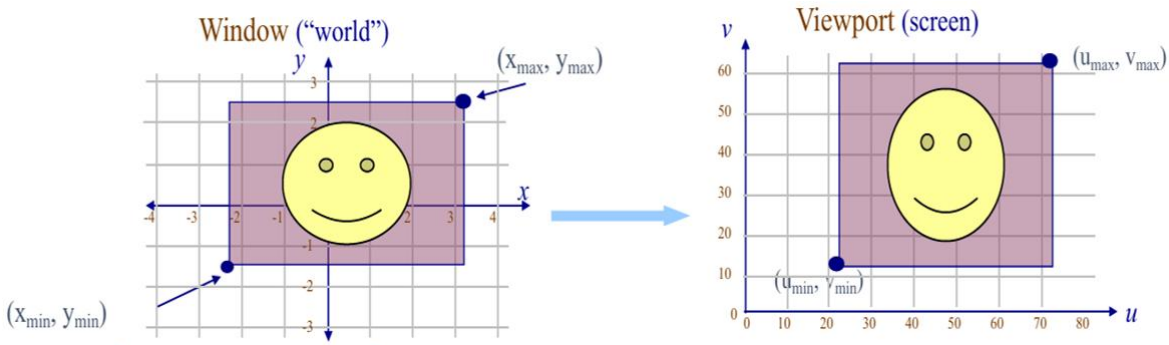
$$S_x = \frac{xv_{max} - xv_{min}}{xw_{max} - xw_{min}}$$

$$S_y = \frac{yv_{max} - yv_{min}}{yw_{max} - yw_{min}}$$



- Step 3: Translate window (position of window must be same as position of view port.





$$t_x = -x_{\min} \qquad t_y = -y_{\min}$$

$$s_x = \frac{u_{\max} - u_{\min}}{x_{\max} - x_{\min}} \qquad s_y = \frac{v_{\max} - v_{\min}}{y_{\max} - y_{\min}}$$

$$t_x = u_{\min} \qquad t_y = v_{\min}$$

$$S_X = \frac{X_{vmax} - X_{vmin}}{X_{wmax} - X_{wmin}}$$

$$S_y = \frac{Y_{vmax} - Y_{vmin}}{Y_{wmax} - Y_{wmin}}$$

The final window-to-viewport transform is:

$$M_{WV} = T(u_{\min}, v_{\min}) S\left(\frac{u_{\max} - u_{\min}}{x_{\max} - x_{\min}}, \frac{v_{\max} - v_{\min}}{y_{\max} - y_{\min}}\right) T(-x_{\min}, -y_{\min})$$

$$= \begin{bmatrix} 1 & 0 & u_{\min} \\ 0 & 1 & v_{\min} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{u_{\max} - u_{\min}}{x_{\max} - x_{\min}} & 0 & 0 \\ 0 & \frac{v_{\max} - v_{\min}}{y_{\max} - y_{\min}} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -x_{\min} \\ 0 & 1 & -y_{\min} \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \frac{u_{\max} - u_{\min}}{x_{\max} - x_{\min}} & 0 & -x_{\min} \frac{u_{\max} - u_{\min}}{x_{\max} - x_{\min}} + u_{\min} \\ 0 & \frac{v_{\max} - v_{\min}}{y_{\max} - y_{\min}} & -y_{\min} \frac{v_{\max} - v_{\min}}{y_{\max} - y_{\min}} + v_{\min} \\ 0 & 0 & 1 \end{bmatrix}$$

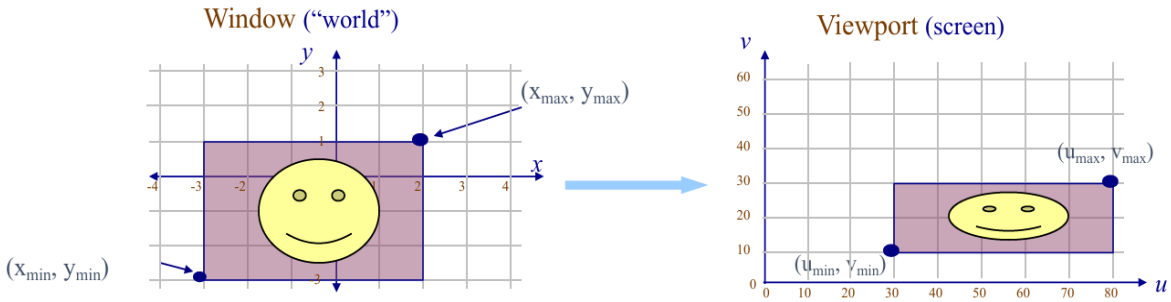
- Multiplying the matrix M_{wv} by the point p gives:

$$p' = \begin{bmatrix} (x - x_{\min}) \frac{u_{\max} - u_{\min}}{x_{\max} - x_{\min}} + u_{\min} \\ (y - y_{\min}) \frac{v_{\max} - v_{\min}}{y_{\max} - y_{\min}} + v_{\min} \\ 1 \end{bmatrix}$$

Window-to-Viewport Example

- Window: $(x_{min}, y_{min}) = (-3, -3)$
  $(x_{max}, y_{max}) = (2, 1)$

- Viewport: $(u_{min}, v_{min}) = (30, 10)$
  $(u_{max}, v_{max}) = (80, 30)$



Window-to-Viewport Example

- Plugging the values into the equation:

$$p' = \begin{bmatrix} (x - x_{min})\dfrac{u_{max} - u_{min}}{x_{max} - x_{min}} + u_{min} \\ (y - y_{min})\dfrac{v_{max} - v_{min}}{y_{max} - y_{min}} + v_{min} \\ 1 \end{bmatrix}$$

$$p' = \begin{bmatrix} (x - (-3))\dfrac{80 - 30}{2 - (-3)} + 30 \\ (y - (-3))\dfrac{30 - 10}{1 - (-3)} + 10 \\ 1 \end{bmatrix} = \begin{bmatrix} (x + 3)\dfrac{50}{5} + 30 \\ (y + 3)\dfrac{20}{4} + 10 \\ 1 \end{bmatrix} = \begin{bmatrix} (x + 3)(10) + 30 \\ (y + 3)(5) + 10 \\ 1 \end{bmatrix}$$

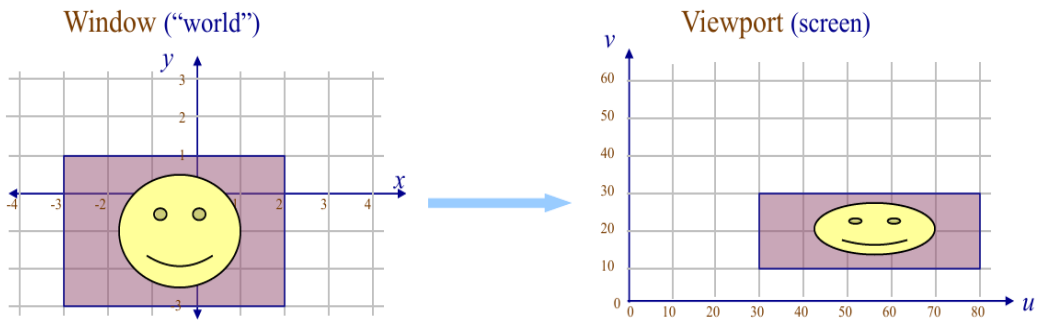
$$= \begin{bmatrix} 10x + 60 \\ 5y + 25 \\ 1 \end{bmatrix}$$

- So:

$$p' = \begin{bmatrix} 10x + 60 \\ 5y + 25 \\ 1 \end{bmatrix}$$

Trying some points:

$(x_{min}, y_{min}) = (-3, -3)$ –>	(30, 10)
$(x_{max}, y_{max}) = (2, 1)$ –>	(80, 30)
Left eye = (-1, -.8) –>	(50, 21)
Top of head = (-0.5, 0.5) –>	(55, 27.5)



CLIPPING

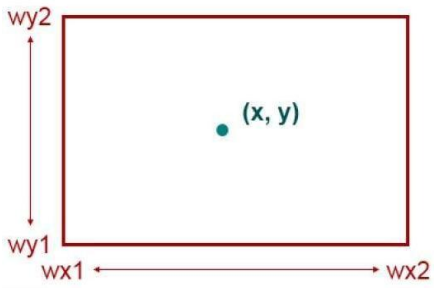
When we must display a large portion of the picture, then not only scaling & translation is necessary, the visible part of picture is also identified. This process is not easy. Certain parts of the image are inside, while others are partially inside. The lines or elements which are partially visible will be omitted.

For deciding the visible and invisible portion, a particular process called clipping is used. Clipping determines each element into the visible and invisible portion. Visible portion is selected. An invisible portion is discarded.

Point Clipping

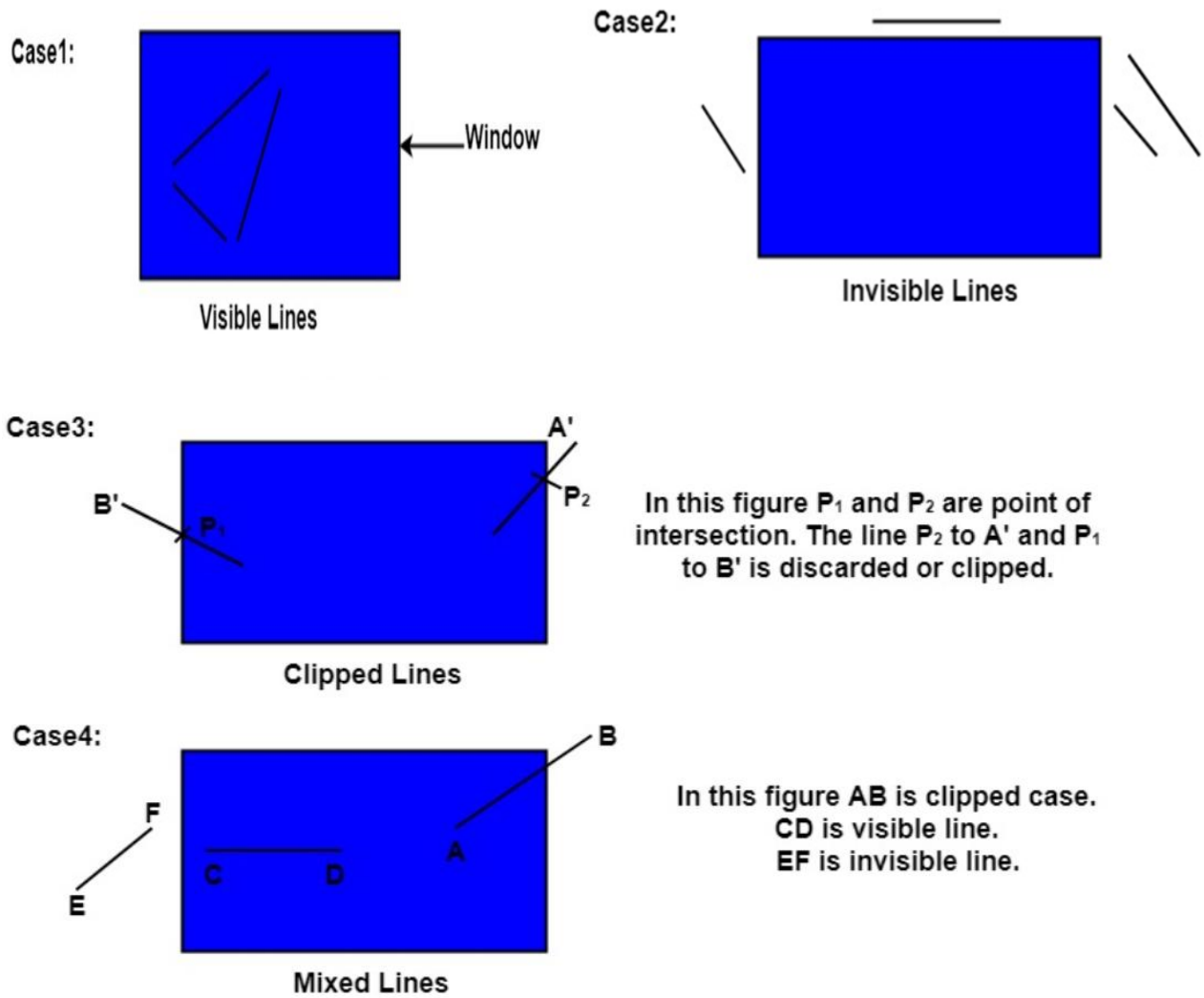
Clipping a point from a given window is very easy. Consider the following figure, where the rectangle indicates the window. Point clipping tells us whether the given point (X, Y) is within the given window or not; and decides whether we will use the minimum and maximum coordinates of the window.

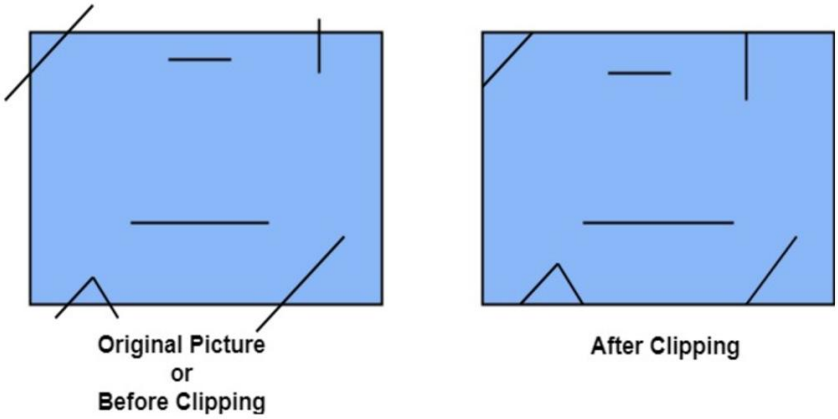
The X-coordinate of the given point is inside the window, if X lies in between $Wx1 \leq X \leq Wx2$. Same way, Y coordinate of the given point is inside the window, if Y lies in between $Wy1 \leq Y \leq Wy2$.



Line Clipping

The concept of line clipping is same as point clipping. In line clipping, we will cut the portion of line which is outside of window and keep only the portion that is inside the window.





- The window against which object is clipped called a clip window.
- It can be curved or rectangle in shape.

Applications of clipping:

- It will extract part we desire.
- For identifying the visible and invisible area in the 3D or 2D object.
- For drawing operations.
- For deleting, copying, moving part of an object.

Types of Clipping:

- Point Clipping,
- **Line Clipping,**
- **Area Clipping (Polygon),**
- Curve Clipping
- Text Clipping,
- Exterior Clipping

Line Clipping Algorithm

1. Cohen Sutherland Line clipping Algorithm
2. Mid-Point Subdivision algorithm

1. Cohen-Sutherland Line Clippings

Most popular line clipping algorithm, speed up the processing of line segments by initial tests that reduce the number of intersections that must be calculated .

All lines come under any one of the following categories

- **Visible:** If a line lies within the window, i.e., both endpoints of the line lie within the window. A line is visible and will be displayed as it is.
- **Not Visible**: If a line lies outside the window, it will be invisible and rejected.

- Clipping Case: If the line is neither visible case nor invisible case. It is the clipped case. Reject the portion of line that are outside the clipping window and accept portion that are inside the clipping window.

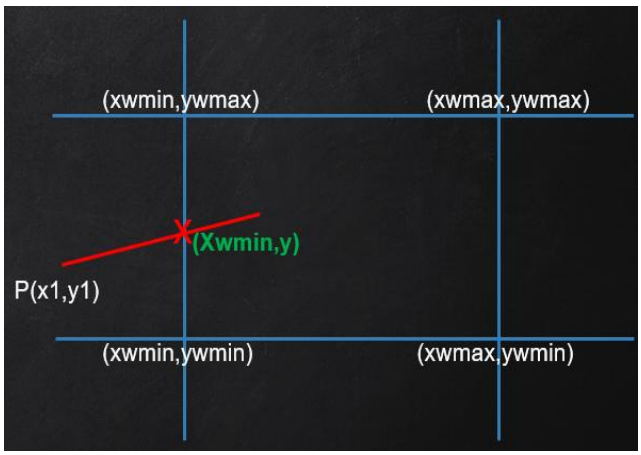
This algorithm uses the clipping window as shown in the following figure. The minimum coordinate for the clipping region is (XWmin, YWmin) and the maximum coordinate for the clipping region is (XWmax, YWmax).

1) **In case of left clipping**, we need to find the intersecting point of line with window boundary

   m = (y-y1) / (xwmin-x1)

   m(xwmin-x1) = (y-y1)

   m(xwmin-x1) +y1= y

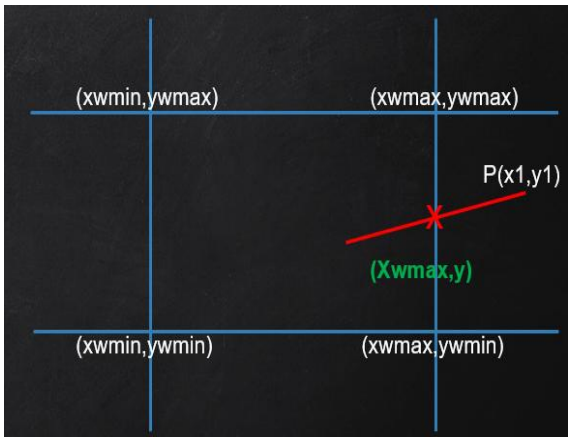


2) **In case of right clipping,** we need to find the intersecting point of line with window boundary

   m = (y-y1) / (xwmax-x1)

   m(xwmax-x1) = (y-y1)

   m(xwmax-x1) +y1= y

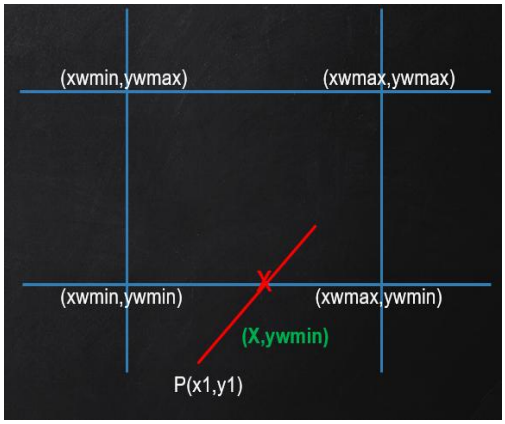


3) **In case of bottom clipping**, we need to find the intersecting point of line with window boundary

m = (ywmin-y1) / (x-x1)

m(x-x1) = (ywmin-y1)

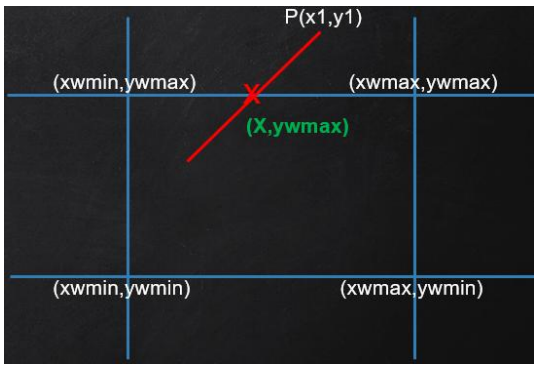
$$X=\frac{(ywmin-y1)}{m}+x1$$



4) **In case of top clipping**, we need to find the intersecting point of line with window boundary

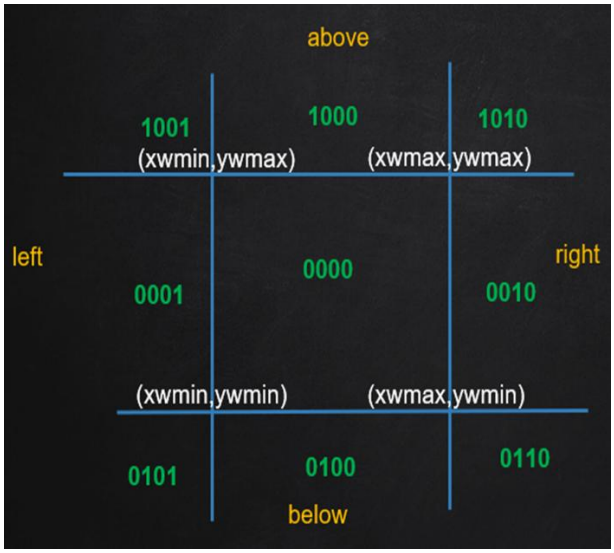
m = (ywmax-y1) / (x-x1)
m(x-x1) = (ywmax-y1)

$$X=\frac{(ywmax-y1)}{m}+x1$$



Every line endpoint in a picture is assigned a four-digit binary code, called a **region code** that identifies the location of the point relative to the boundaries of the clipping window. Regions are set up in reference to the boundaries of the clipping window.

We will use 4-bits to divide the entire region. These 4 bits represent the Top, Bottom, Right, and Left of the region as shown in the following figure. Here, the TOP and LEFT bit is set to 1 because it is the TOP-LEFT corner.



bit4	bit3	bit2	bit1
above	below	right	left

```
If x is less than x_min
            then bit number 1 is set.

 If x is greater than x_max
            then bit number 2 is set.

If y is less than y_min
            then bit number 3 is set.

 If y is greater than y_max
           then bit number 4 is set
```

There are 3 possibilities for the line −

- Line can be completely inside the window (This line should be accepted).
- Line can be completely outside of the window (This line will be completely removed from the region).
- Line can be partially inside the window (We will find intersection point and draw only that portion of line that is inside region).

(a) If bit 1 is "1" line intersects with left boundary of rectangle window

$$y=y_1+m(x-x_1)$$

where $x = x_{wmin}$

where x_{wmin} is the minimum value of X co-ordinate of window ,(x1,y1) line endpoint coordinate .

(b) If bit 2 is "1" line intersecting with right boundary

$$y=y_1+m(x-x_1)$$

where $x = x_{wmax}$

where x more is maximum value of x co-ordinate of the window, (x1,y1) line endpoint coordinate .

(c) If bit 3 is "1" line intersects with bottom boundary

$$X=X1+(y-y_1)/m$$

where $y = y_{wmin}$

y_{wmin} is the minimum value of Y co-ordinate of the window ,(x1,y1) line endpoint coordinate .

(d) If bit 4 is "1" line intersects with the top boundary

$$X =X1+ (y-y_1)/m$$

where $y = y_{wmax}$

y_{wmax} is the maximum value of Y co-ordinate of the window ,(x1,y1) line endpoint coordinate .

Algorithm

Step 1: Assign a region code for two endpoints of given line.

Step 2: If both endpoints have a region code 0000 then given line is completely inside.

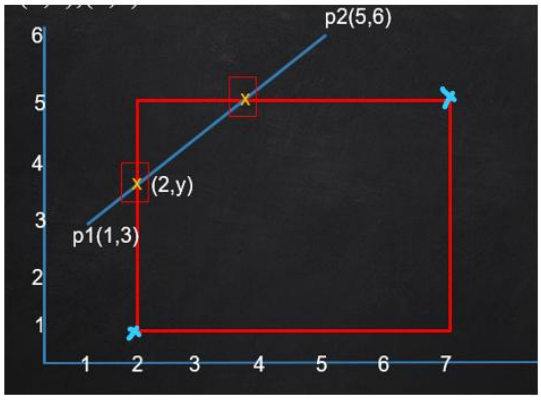
Step 3: Else, perform the logical AND operation for both region codes.

- Step 3.1: If the result is not 0000, then given line is completely outside.
- Step 3.2: Else line is partially inside.
  - Step 3.2.1: Choose an endpoint of the line that is outside the given rectangle.
  - Step 3.2.2: Find the intersection point of the rectangular boundary.
  - Step 3.2.3: Replace endpoint with the intersection point and update the region code.
  - Step 3.2.4: Repeat step 2 until we find a clipped line either trivially accepted or trivially rejected.

Step 4: Repeat step 1 for other lines.

Problem solving

Clipping window lower left(2,1) , upper right(7,5)
Line points (1,3),(5,6)



P1 region code:0001
P2 region code:1000

P1 AND P2
0001
1000
0 0 0 0

P1 intersecting Left edge

$y=y_1+m(x-x_1)$

$x = x_{wmin}$

m = (6-3)/(5-1)
= 3/4=0.75

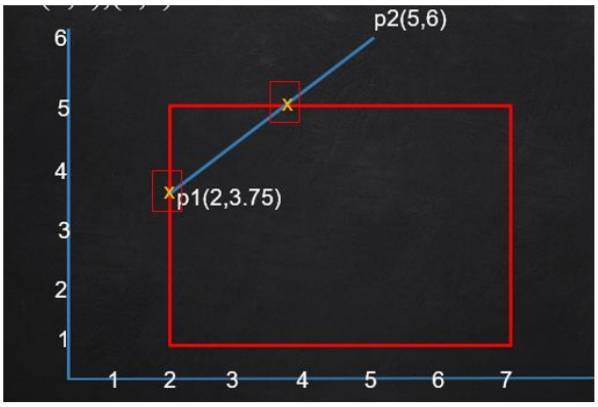
X=2

y=3+0.75(2-1)
=3.75

(x,y)=(2,3.75) -- new end point p1

P1 New region code 0000 ,completely inside

Clipping window lower left(2,1) , upper right(7,5)
Line points (1,3),(5,6)



P1 region code:0000
P2 region code:1000

P1 AND P2
0000
1000
0000

P2 intersecting above edge

$X=X1+(y-y_1)/m$

where $y = y_{wmax}$

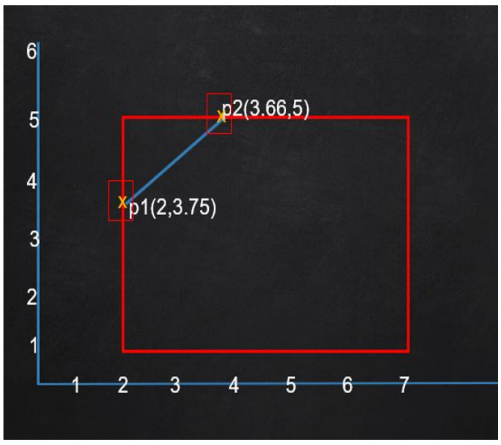
m = (6-3)/(5-1)
= 3/4=0.75

y=5

x=5+(5-6) /.75
=5 - (1.33)= 3.66

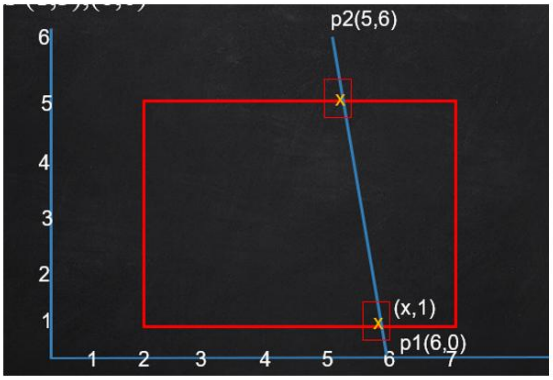
(x,y)=(3.66,5) new end point p2

P2 New region code 0000



P1 and p2 region codes 0000 ,
Completely accept now and stop .

2Q) Clipping window lower left(2,1) , upper right(7,5)
Line points (5,6) (6,0)



P1 New region code 0000 , (x, y)=(5.83,1) new end point p1

P1 region code:0100
P2 region code:1000

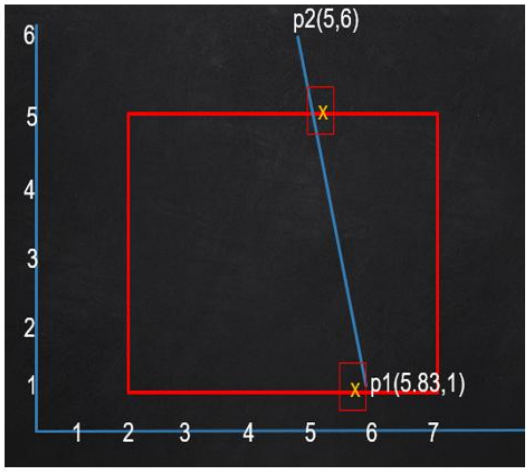
P1 AND P2
0100
1000
0000

P1 intersecting below edge
$x = x_1 + (y - y_1)/m$
where $y = y_{wmin}$

y=1 m = (6-0)/(5-6)
= 6/-1= - 6

X= 6+(1-0) / - 6 = 6 - 0.1667
= 5.83

Clipping window lower left(2,1) , upper right(7,5)
Line points (1,3),(6,0)



P1 region code:0000
P2 region code:1000
P1 AND P2
0000
1000
0000

P2 intersecting above edge
X=X1+(y-y1)/m

where y = ywmax

m = (6-0)/(5-6)
= 6/-1=-6

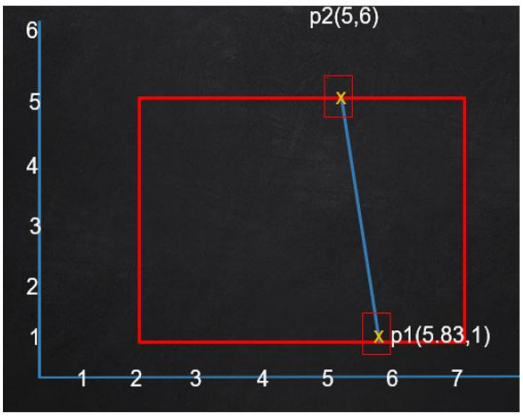
Y=5

X = 5+(5-6)/-6 =-1/-6
= 5+0.167 = =5.167

(x, y)=(5.167,5) --new end point p2

P2 New region code 0000 ,

Clipping window lower left(2,1) , upper right(7,5)
Line points (1,3),(6,0)



P1 and p2 region codes 0000 ,
Completely accept now and stop .

Example 5.1 Given a clipping window — A(20,20) B(60,20) C(60,40), D(20,40). Using Sutherland Cohen algorithm find the visible portion of line segment joining the points P(40,80) Q(120,30).

Solution

End point code of P is (1000)

End point code of Q is (0010)

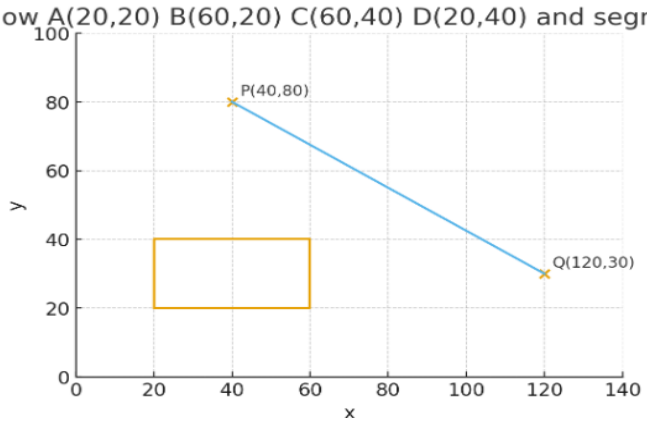
Both end point codes are not zero, and their logical AND is zero. Hence, line cannot be rejected a invisible.

- Slope of PQ = $m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{30-80}{120-40} = \frac{-50}{80} = \frac{-5}{8}$

  $[(x_1,y_1) = (40,80) ; (x_2,y_2) = (120,30)]$

  and intersections with window edge are -

Window A(20,20) B(60,20) C(60,40) D(20,40) and segment P-Q



Intersections of the infinite line with window edges:

1) With right edge x=60 and (X1,Y1) = (40,80)

$y=y_1+m(x-x_1)$

where $x = x_{wmax}$

x=60:

$y = 80+[\frac{-5}{8}(60-40)] = 80 + [\frac{(-5 * 20)}{8}] = 80 - 12.5 = 67.5$

y=80+m(60−40)=67.5 (this is above the window top y=40 → intersection outside the window edge segment).

2)With top edge y=40 and (X1,Y1) = (40,80)

$X = X1 + (y-y_1)/m$

where $y = y_{wmax}$

Y= 40

$X = 40 + [\frac{(40-80) * -8}{5}] = 40 + [320/5] = 40 + 64 = 104$

(this is to the right of the window x=60 → outside).

3) With left edge x=20 and (X1, Y1) = (120,30)

x=20

y=y1+m(x-x1)

where x = xwmin
y= 30 + (20-120) * (-5/8)
= 30+ [(100*5)/8] = 30+62.5
=92.5
y=92.5 (above top → outside).

4) With bottom edge *y*=20 and (X1, Y1) = (120,30)
y=20:
X=X1+(y-y1)/m

where y = ywmin
X= 120 +(20-30) * (-8/5) = 120 + [-10 * -8/5] =120 + 16 = 136
x=136 (which to the right → outside).

So, there is no parameter t on the segment that gives a point inside the window.
Therefore, although the out codes' AND is zero (so not trivially rejected), the segment never actually passes through the rectangular window — the whole segment lies above the window for the x-range that would enter the window.
Result: the line segment P–Q has no visible portion inside the window (it is completely outside and should be rejected

2. Midpoint Subdivision algorithm

The strength of this algorithm over the Cohen-Sutherland algorithm is that it requires no floating-point arithmetic to find the point of intersection with the line and the clip boundary. The midpoint subdivision algorithm clips a line by finding the endpoints of the visible portion of the line segment. Each endpoint can be found by an identical process and given appropriate hardware, this can be done in parallel for both endpoints.

Also, the Sutherland Cohen subdivision line clipping algorithm requires the calculation of the intersection of the line with the window edge. These calculations can be avoided by repetitively subdividing the line at its midpoint.

Like other algorithm, initially the line is tested for visibility. If line is completely visible it is drawn and if it is completely invisible it is rejected. If line is partially visible, then it is subdivided in two equal parts. The visibility tests are then applied to each half. This subdivision process is repeated until we get completely visible and completely invisible line segments. This is illustrated in figure (k) below

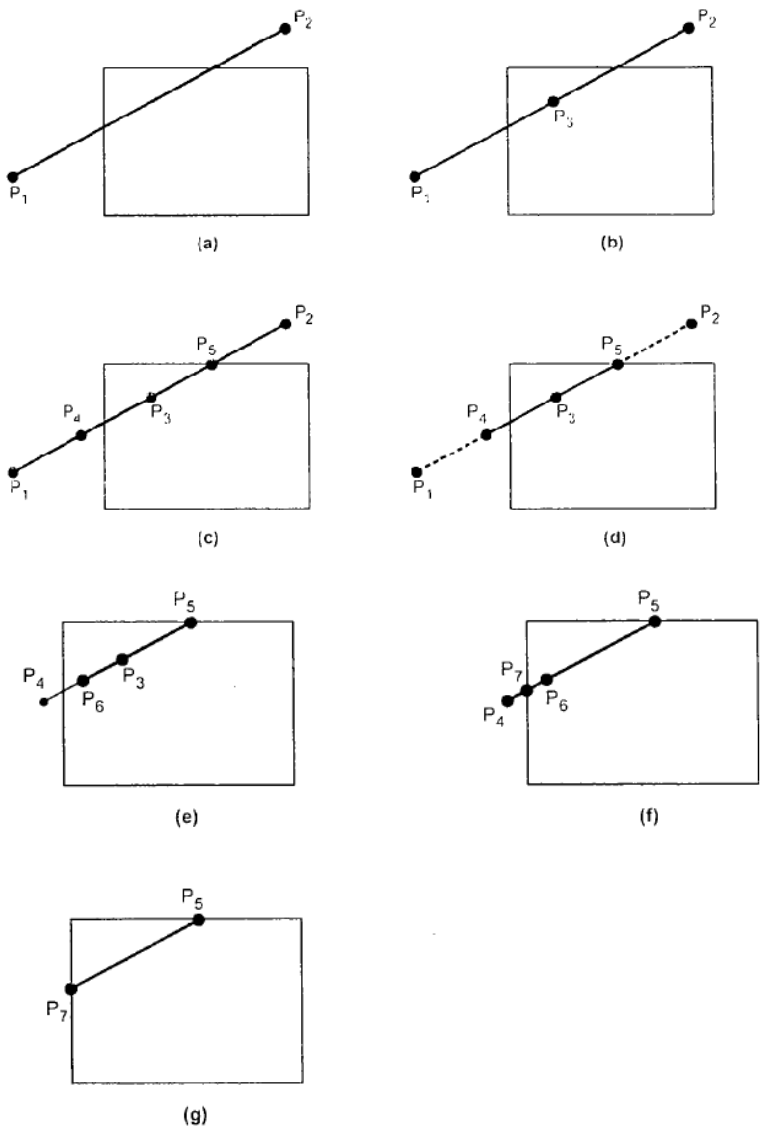


Fig. (k) Clipping line with midpoint subdivision algorithm

As shown in the figure (k), line P1 P2 is partially visible. It is subdivided in two equal Parts P1 P3 and P3 P2 (see Fig. k (b)). Both the line segments are tested for visibility and found to be partially visible. Both line segments are then subdivided in two equal parts to get midpoints P4 and P5 (see Fig. k (c)). It is observed that line segments P1 P4 and P5 P2 are completely invisible and hence rejected. However, line segment P3 P5 is completely visible and hence drawn. The remaining line segment P4 P3 is still partially visible. It is then subdivided to get midpoint P6. It is observed that P6 P3 is completely visible whereas P4 P6 is partially visible. Thus, P6 P3 line segment is drawn and P4 P6 line segment is further subdivided into equal parts to get midpoint P7. Now, it is observed that line segment P4 P7 is completely invisible and line segment P7 P6 is completely visible (see Fig. k (f)), and there is no further partially visible segment.

Midpoint Subdivision Algorithm :

1. Read two endpoints of the line say $P_1(x_1, y_1)$ and $P_2(x_2, y_2)$.
2. Read two corners (left-top and right-bottom) of the window, say (Wx_1, Wy_1 and Wx_2, Wy_2).
3. Assign region codes for two end points using following steps :
   Initialize code with bits 0000
   Set Bit 1 - if ($x < Wx_1$)
   Set Bit 2 - if ($x > Wx_2$)
   Set Bit 3 - if ($y < Wy_1$)
   Set Bit 4 - if ($y > Wy_2$)
4. Check for visibility of line
   a) If region codes for both endpoints are zero then the line is completely visible. Hence draw the line and go to step 6.
   b) If region codes for endpoints are not zero and the logical ANDing of them is also nonzero then the line is completely invisible, so reject the line and go to step 6.
   c) If region codes for two endpoints do not satisfy the conditions in 4a) and 4b) the line is partially visible.
5. Divide the partially visible line segment in equal parts and repeat steps 3 through 5 for both subdivided line segments until you get completely visible and completely invisible line segments.
6. Stop.

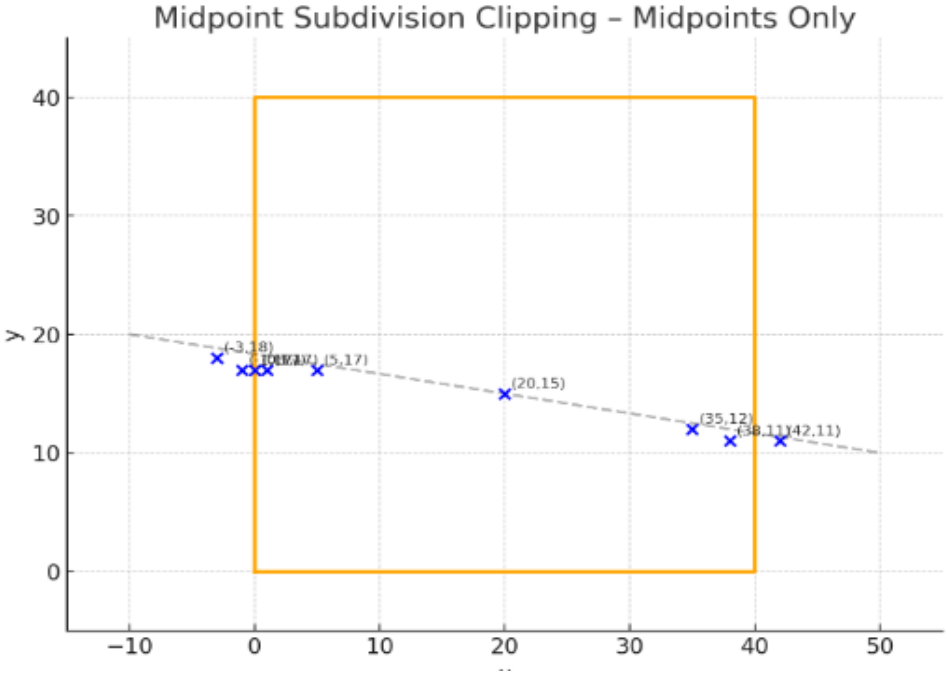
Q) Given a clipping window A (0,0), B (40,0), C(40,40),D(0,40). Using the Midpoint Subdivision algorithm, find the visible portion of the line segment joining the points P (−10,20) and Q (50,10)

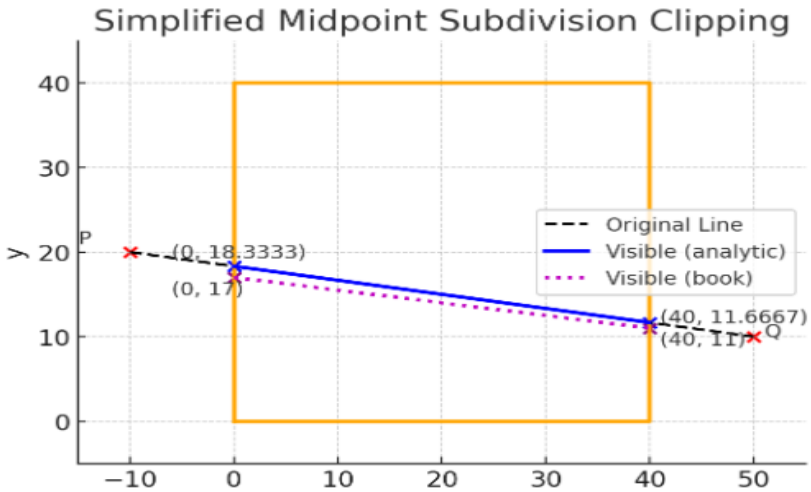
Solution End-point codes of point P is (0001) and that of point Q is (0010). Both endpoint codes are not zero, and their logical AND is zero, hence we can conclude that line cannot be rejected as invisible.
Midpoint is

$$x_m = \frac{x_2 + x_1}{2} = \frac{50 + (-10)}{2} = 20$$

$$y_m = \frac{y_2 + y_1}{2} = \frac{10 + 20}{2} = 15$$

End point code for midpoint $P_m(x_m, y_m)$ is (0000). Neither segment PP_m nor P_mQ is either totally visible or trivially invisible. Lets keep segment PP_m for later processing, and we continue with P_mQ.


Midpoint Subdivision Clipping – Midpoints Only
y
(-3,18)
(5,17)
(20,15)
(35,12)
(38,11)
(42,11)


Simplified Midpoint Subdivision Clipping
y
P
(0, 18.3333)
(0, 17)
(40, 11.6667)
(40, 11)
Q
Original Line
Visible (analytic)
Visible (book)

P	Q	P_m	*Comment*
(–10,20)	(50,10)	(20,15)	Save PP_m, and continue with P_mQ.
(20, 15)	(50,10)	(35,12)	Continue with P_mQ, as PP_m is totally visible.
(35,12)	(50,10)	(42,11)	Continue with PP_m as P_mQ is totally invisible.
(35,12)	(42,11)	(38,11)	Continue with P_mQ.
(38,11)	(42,11)	(40,11)	Success. This is the intersection point of line with right window edge.
(–10,20)	(20,15)	(5,17)	Recall saved PP_m, Continue with PP_m, as P_mQ is totally visible.
(–10,20)	(5,17)	(–3,18)	Continue with P_mQ, as PP_m is totally invisible.
(–3,18)	(5,17)	(1,17)	Continue with PP_m.
(–3,18)	(1,17)	(–1,17)	Continue with P_mQ.
(–1,17)	(1,17)	(0,17)	Success. This is the intersection point of line with left window edge.

Conclusion: Visible portion of line segment PQ is from (0,17) to (40,11).

[Note: Midpoint algorithm uses integer functions, which works like floor functions.]

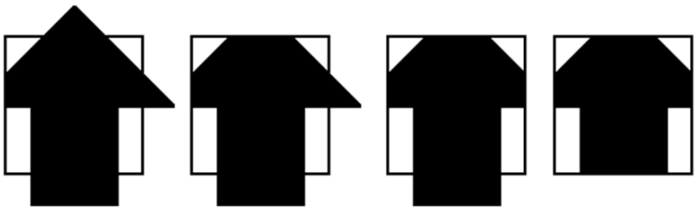
SUTHERLAND - HODGMAN POLYGON CLIPPING

The Sutherland - Hodgman algorithm performs a clipping of a polygon against each window edge in turn. It accepts an ordered sequence of vertices v1, v2, v3, ..., vn and puts out a set of vertices defining the clipped polygon.

Before clipping

This figure represents a polygon (the large, solid, upward pointing arrow) before clipping has occurred.

The following figures show how this algorithm works at each edge, clipping the polygon.

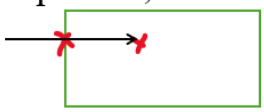


(a)((b) (c) (d)

a. Clipping against the left side of the clip window.
b. Clipping against the top side of the clip window.
c. Clipping against the right side of the clip window.

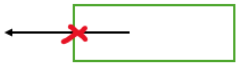
d. Clipping against the bottom side of the clip window.

- If the first vertex is an outside the window, the second vertex is inside the window. The point of intersection of window boundary and polygon side (edge) is added to the output list, second vertex is also added to the output list. {outside to inside}.



- If both vertexes are inside window boundary. Then only second vertex is added to the output list .{ inside to inside}.

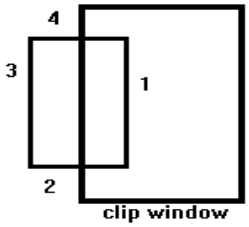
- If the first vertex is inside the window and second is an outside window. The edge which intersects with window is added to output list.{inside to outside }.



- If both vertices are the outside window, then nothing is added to output list.

Four Types of Edges

As the algorithm goes around the edges of the window, clipping the polygon, it encounters four types of edges. All four edge types are illustrated by the polygon in the following figure. For each edge type, zero, one, or two vertices are added to the output list of vertices that define the clipped polygon.



The four types of edges are:

1. Edges that are totally inside the clip window. - add the second inside vertex point
2. Edges that are leaving the clip window. - add the intersection point as a vertex
3. Edges that are entirely outside the clip window. - add nothing to the vertex output list
4. Edges that are entering the clip window. - Save the intersection and inside points as vertices.

Disadvantage: The algorithm correctly clips convex polygons but may display extraneous lines for concave polygons. So, this algorithm is not suited for concave polygons.

Example:

We illustrate this algorithm by processing the area in figure against the left window boundary.

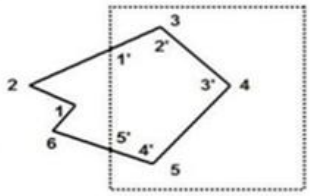
Vertices 1 and 2 are outside of the boundary.

Vertex 3, which is inside, 1' and vertex 3 are saved.

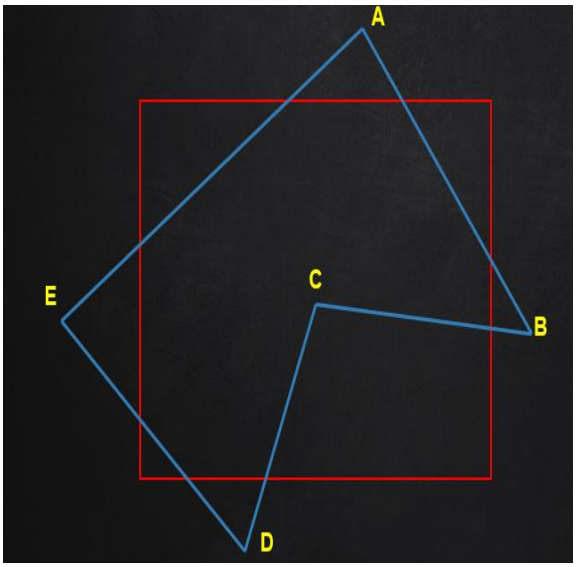
Vertex 4 and 5 are inside, and they also saved.

Vertex 6 is outside, 5' is saved.

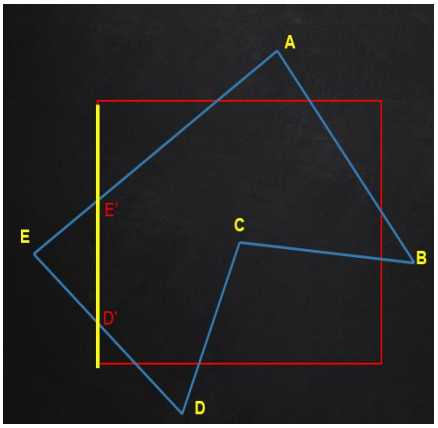
Using the five saved points, we would repeat the process for the next window boundary.



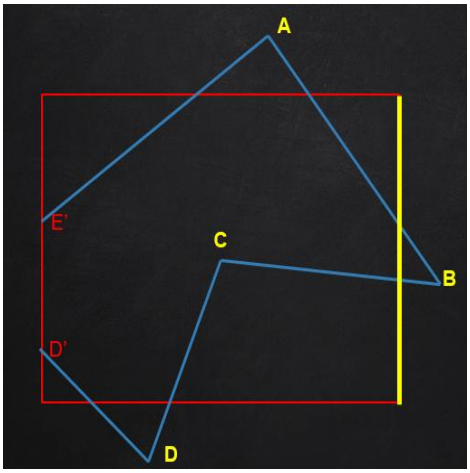
Q1) Perform sutherland - hodgman polygon clipping on the following polygonal.



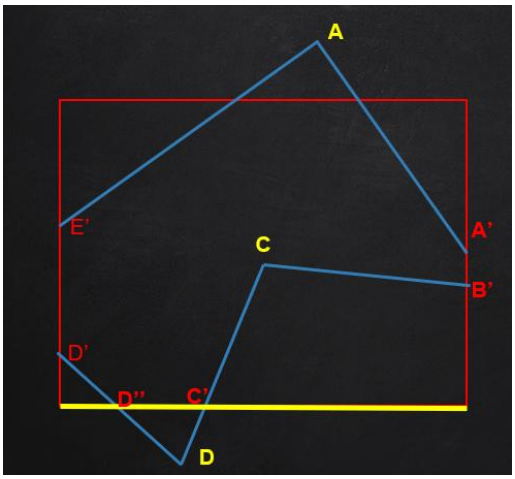
Solution:-



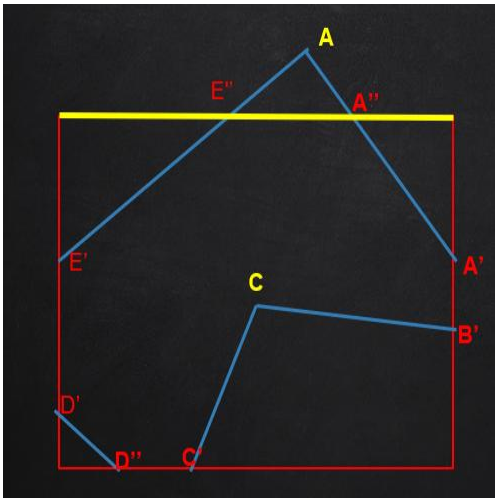
VERTEX	CASE	OUTPUT
AB	In→ in	B
BC	In → in	C
CD	In → in	D
DE	In → out	D'
EA	Out →in	E'A



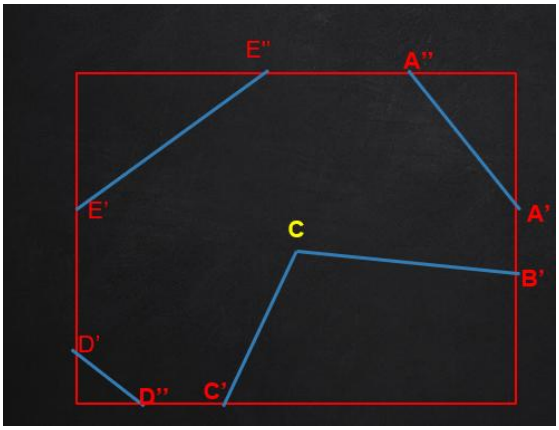
VERTEX	CASE	OUTPUT
AB	In → out	A'
BC	Out → in	B'C
CD	In → in	D
DD'	In → in	D'
D'E'	In → in	E'
E'A	In → in	A



VERTEX	CASE	OUTPUT
AA'	in→in	A'
A'B'	in→in	B'
B'C	in→in	C
CD	in→out	C'
DD'	out→in	D'' D'
D'E'	in→in	E'
E'A	in→in	A



VERTEX	CASE	OUTPUT
AA'	out→in	A'' A'
A'B'	in→in	B'
B'C	in→in	C
CC'	in→in	C'
C'D''	in→in	D''
D''D'	in→in	D'
D'E'	in→in	E'
E'A	in→out	E''

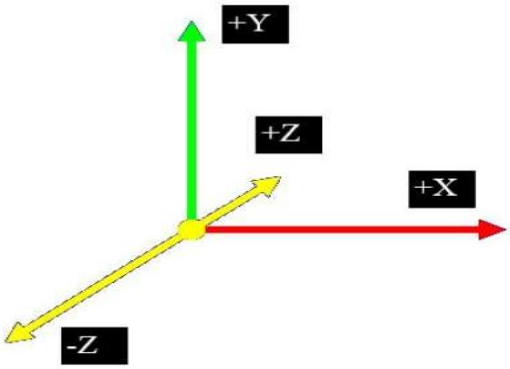


PROJECTION

Three-dimensional viewing pipeline. Projections- Parallel and Perspective projections. Visible surface detection algorithms- Depth buffer algorithm, Scan line algorithm

In the 2D system, we use only two coordinates X and Y but in 3D, an extra coordinate Z is added. 3D graphics techniques and their application are fundamental to the entertainment, games, and computer-aided design industries. It is a continuing area of research in scientific visualization.

3D graphics components are now a part of almost every personal computer and, although traditionally intended for graphics-intensive software such as games, they are increasingly being used by other applications.

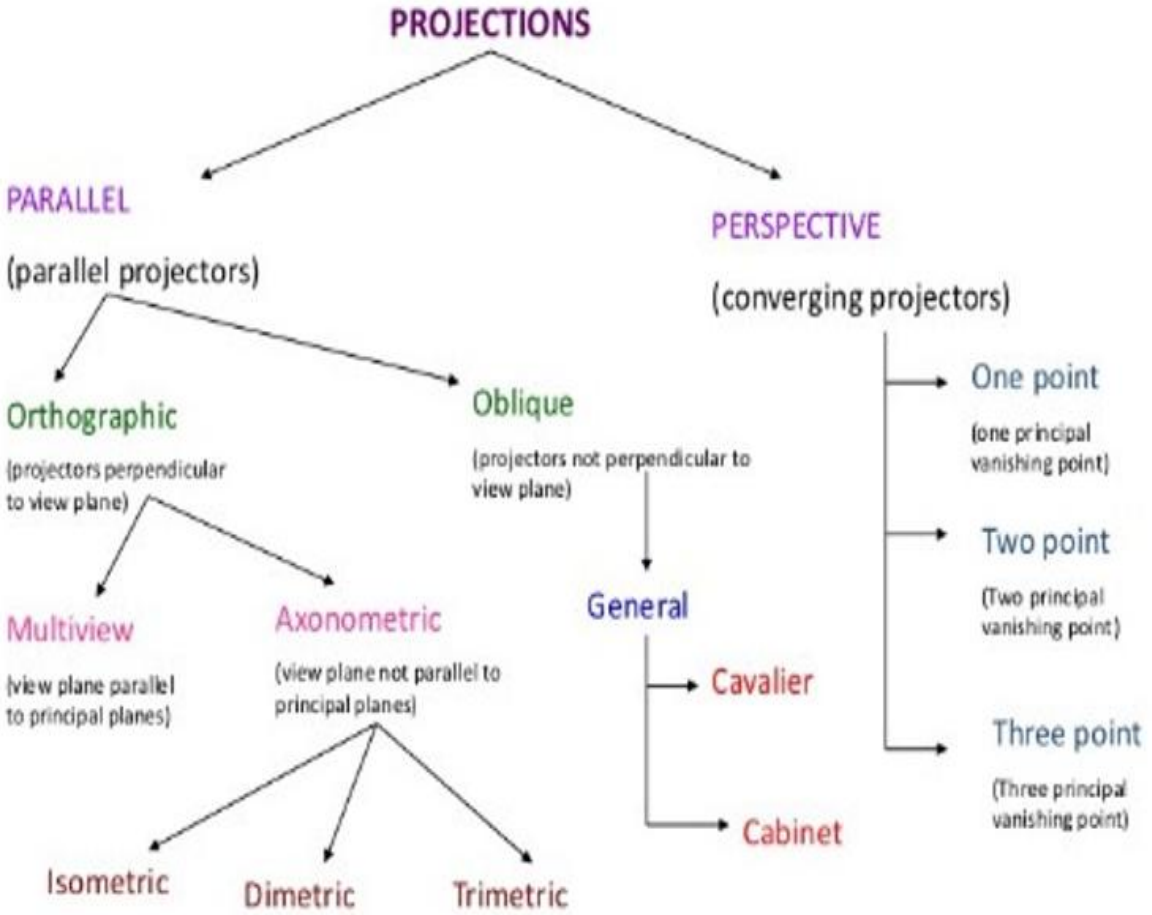


PROJECTION

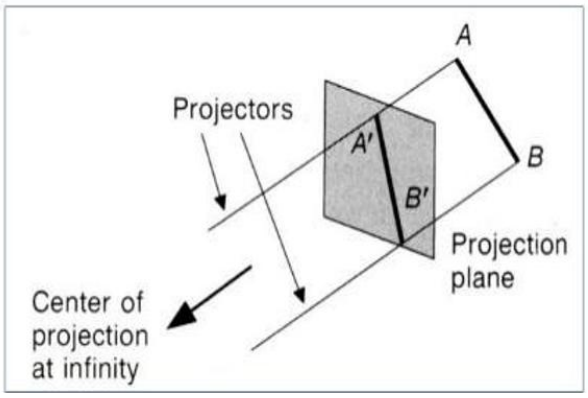
- Projection can be defined as mapping a point P(x,y,z) onto its image P'(x',y',z') in the projection plane or view plane.
- Projection Transforms 3D objects on to a 2D plane
- **2 types of projections**
  - Perspective Projection
  - Parallel Projection

Parallel Projection: Coordinate positions are transformed to the view plane along parallel lines.

Perspective projection: Object position is transformed to the view plane along lines that converge to a point called **projection reference point** (center of projection)



Parallel Projection

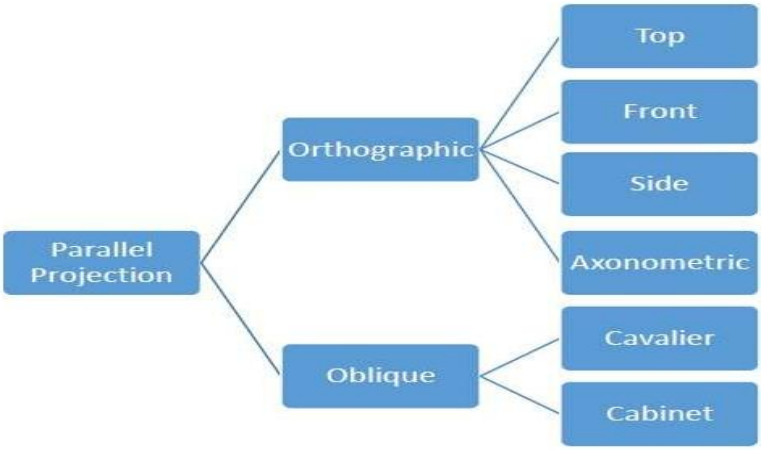


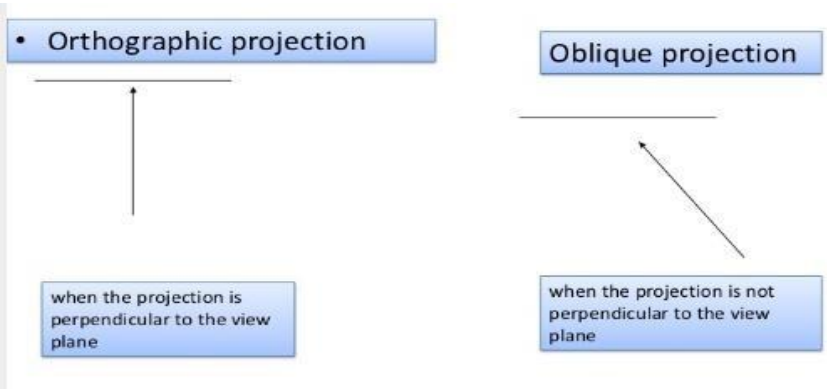
- Center of Projection is located at infinity; all the projectors are parallel
- Parallel projection is used in drawing objects when perspective projection cannot be used.
- Parallel projection is much like seeing objects through a telescope, letting parallel light rays into the eyes which produce visual representations without depth
- Parallel projection may be best for architectural drawings, in cases

wherein measurements are necessary

- For parallel projections, we specify a direction of projection (DOP) instead of a COP
- There are two types of parallel projections:

1. **Orthographic projection — DOP perpendicular to PP**
2. **Oblique projection — DOP not perpendicular to PP**





1. **<u>Orthographic Projection</u>**

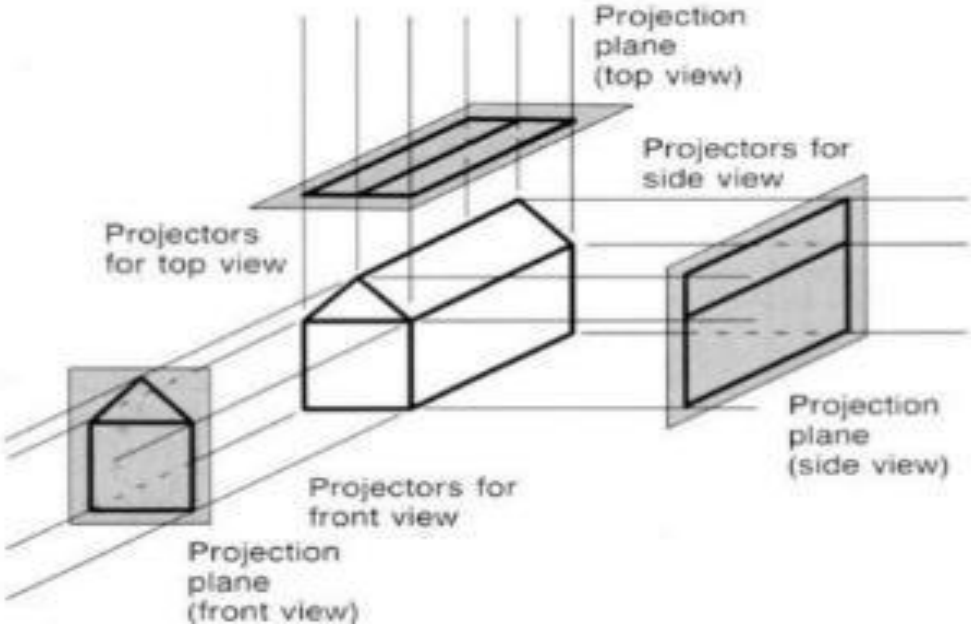
In orthographic projection the direction of projection is normal to the projection of the plane.

- Front, side and rear orthographic projection of an object are called elevations
- The top orthographic projection is called plan view
- All have projection plane perpendicular to a principle axes.
- Length and Angles are accurately depicted and measured from the drawing
- Engineering and architectural drawings commonly employee this

1. **<u>MULTIVIEW PROJECTION</u>**

There are three types of orthographic projections –

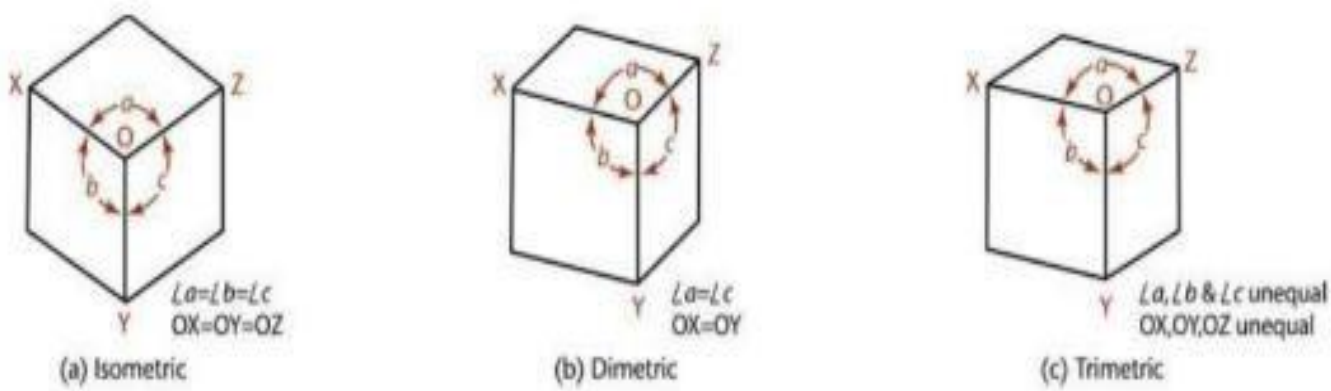
- Front Projection
- Top Projection (Plane View)
- Side Projection



2. AXONOMETRIC PROJECTION

- Orthographic projections that show more than one face of an object are called **axonometric orthographic projections**
- The most common is isometric projection
- Projection plane intersects each coordinate axis in the model coordinate system at an equal distance

1. **Isometric projection** - The direction of projection makes equal angles with all three principal axes.
2. **Dimetric projection**- The direction of projection makes equal angles with exactly two of the principal axes.
3. **Trimetric projection-** The direction of projection makes unequal angles with the three principal axes.



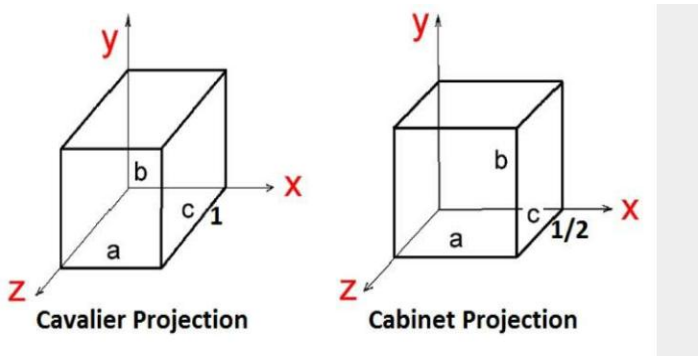
3. Oblique Projection

In oblique projection, the direction of projection is not normal to the projection of plane. In oblique projection, we can view the object better than orthographic projection.

There are two types of oblique projections − **Cavalier and Cabinet**.

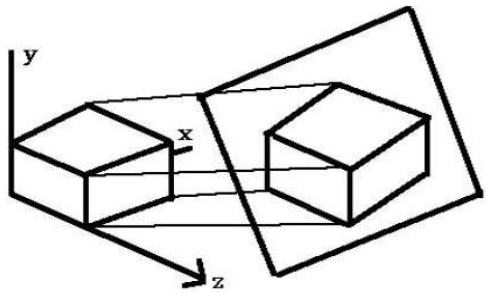
The **Cavalier projection** makes 45° angle with the projection plane. The projection of a line perpendicular to the view plane has the same length as the line itself in Cavalier projection. In a cavalier projection, the foreshortening factors for all three principal directions are equal.

The **Cabinet projection** makes 63.4° angle with the projection plane. In Cabinet projection, lines perpendicular to the viewing surface are projected at ½ their actual length. Both the projections are shown in the following figure –

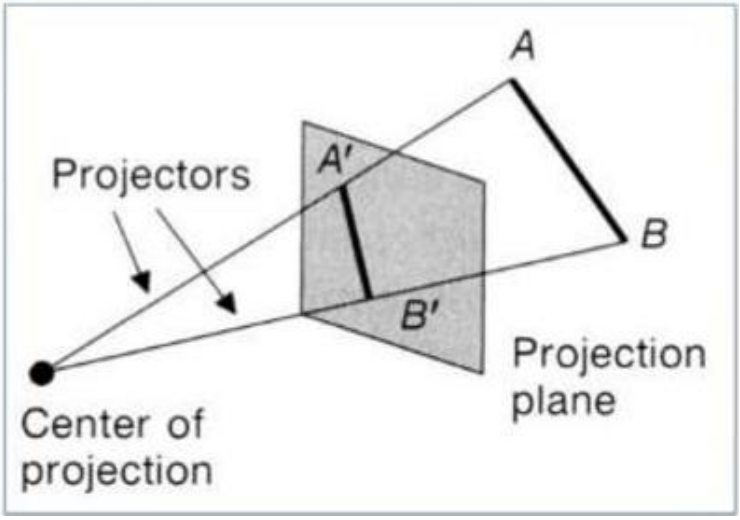


Isometric Projections

Orthographic projections that show more than one side of an object are called axonometric orthographic projections. The most common axonometric projection is an isometric projection where the projection plane intersects each coordinate axis in the model coordinate system at an equal distance. In this projection parallelism of lines are preserved but angles are not preserved. The following figure shows isometric projection –



Perspective Projection



- Center of projection is located at a finite point
- Perspective projection is representing or drawing objects which resemble the real thing
- Perspective projection represents objects in a three-dimensional way.
- In perspective projection, objects that are far away appear smaller, and objects that are near appear bigger
- Perspective projections require a distance between the viewer and the target point.
    - **Types:**
      1. **one point perspective**
      2. **Two-point perspective**
      3. **Three-point perspective**

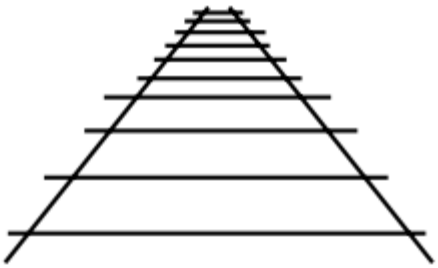
<u>Perspective Projection</u>

In perspective projection, the distance from the center of projection to project plane is finite and the size of the object varies inversely with distance which looks more realistic.

The distance and angles are not preserved, and parallel lines do not remain parallel. Instead, they all converge at a single point called center of projection or projection reference point.

Vanishing point

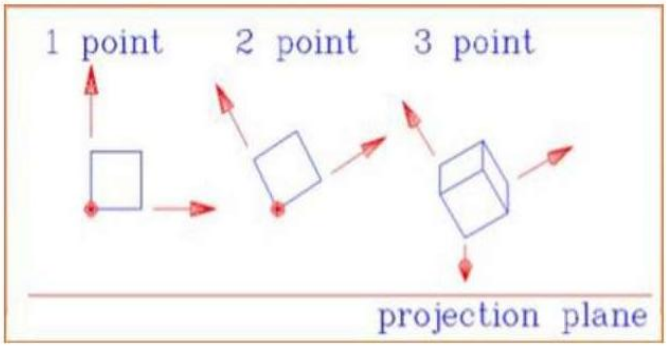
- Under perspective projections, any set of parallel lines that are not parallel to the PP will converge to a vanishing point
- <u>Vanishing points of lines parallel to a principal axis x, y, or z are called principal vanishing points.</u>



Perspective drawings are often classified by the number of principal vanishing points

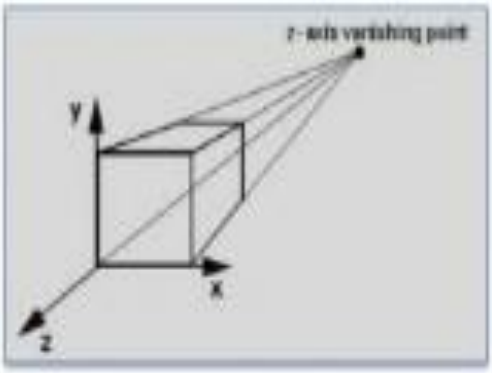
There are 3 types of perspective projections which are shown in the following chart.

1. **One point perspective projection is simple to draw.**
2. **Two-point perspective projection gives better impression of depth.**
3. **Three-point perspective projection is most difficult to draw.**



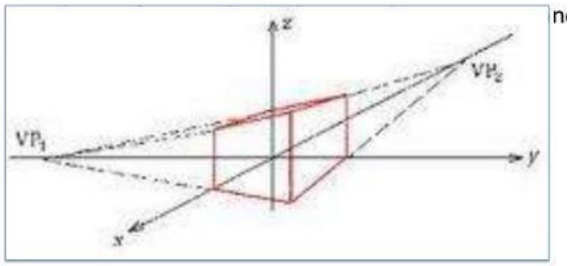
One point Perspective

- Simplest to draw
- One principal face parallel to projection plane
- One vanishing point



<u>Two-point Perspective</u>

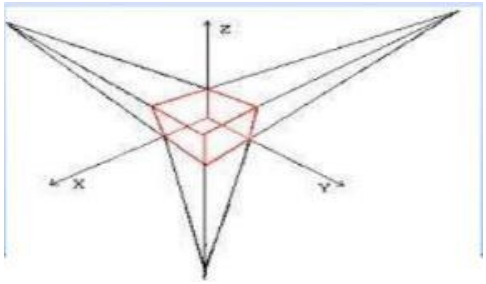
o Two vanishing points



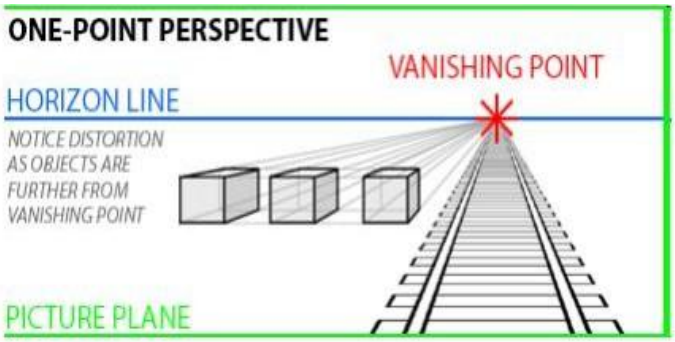
Three-point Perspective

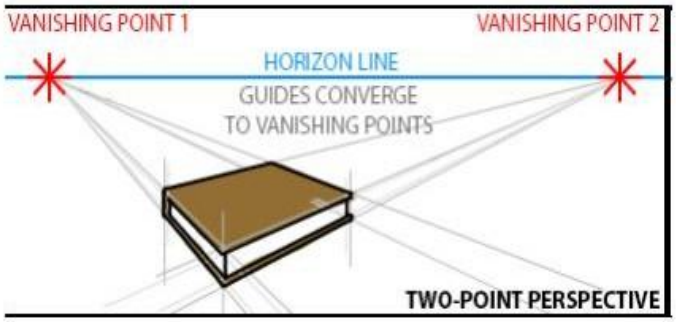
o No principal face parallel to projection plane

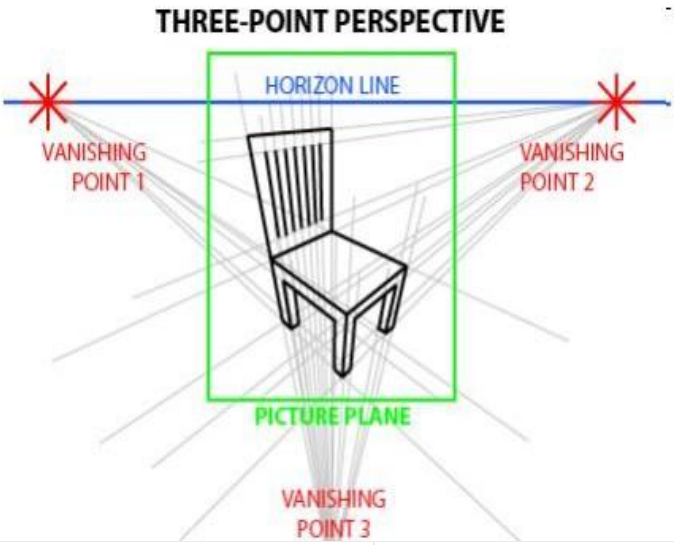
o Three vanishing points



More examples





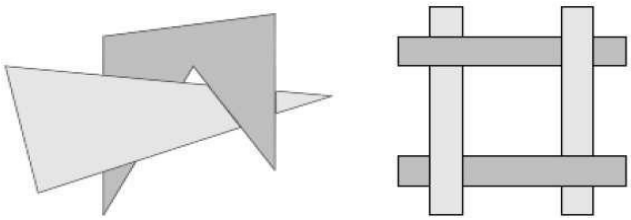


S.NO.	**PERSPECTIVE PROJECTIONS**	**PARALLEL PROJECTION**
1.	If COP [Centre Of Projection] is located at a finite point in 3 space, the result is a perspective projection.	If COP [Centre of Projection] is located at infinity, all the projectors are parallel, and the result is a parallel projection.
2.	Perspective projection is representing or drawing objects which resemble the real thing	parallel projection is used in drawing objects when perspective projection cannot be used.
3.	perspective projection represents objects in a three-dimensional way.	Parallel projection is much like seeing objects through a telescope, letting parallel light rays into the eyes which produce visual representations without depth
4.	In perspective projection, objects that are far away appear smaller, and objects that are near appear bigger	parallel projection does not create this effect.
5.	While parallel projection may be best for architectural drawings, in cases wherein measurements are necessary	it is better to use perspective projection.

6.	perspective projections require a distance between the viewer and the target point.	In parallel projection the center of projection is at infinity, while in prospective projection, the center of projection is at a point.
7.	Types: 1.one point perspective, 2.Two point perspective, 3. Three point perspective,	Types: 1.Orthographic 2.Oblique

Visible Surface Detection

When we view a picture containing non-transparent objects and surfaces, then we cannot see those objects from view which are behind from objects closer to eye. We must remove these hidden surfaces to get a realistic screen image. The identification and removal of these surfaces is called **Hidden-surface problem.**



- When you draw a scene composed of three-dimensional objects, some of them might obscure all or parts of others.
- Changing your viewpoint can change the obscuring relationship.
- For example, if you view the scene from the opposite direction, any object that was previously in front of another is now behind it.
- To draw a realistic scene, these obscuring relationships must be maintained.

There are two approaches for removing hidden surface problems
1.Object-Space method
2. Image-space method.

1. **Object Space Methods**

- These methods examine objects, faces, edges etc. to determine which are visible.
- The complexity depends upon the number of faces, edges etc. in all the objects.
- The Object-space method is implemented in physical coordinate system and image-space method is implemented in screen coordinate system.

Compare objects and parts of objects to each other within the scene definition to determine which surfaces, as a whole, we should label as visible:

For each object in the

scene do Begin

1. Determine those parts of the object whose view is unobstructed

by other parts of it or any other object with respect to the viewing specification.

2. Draw those parts in the object color.

End

2. <u>Image Space Methods</u>

- These methods examine each pixel in the image to determine which face of which object should be displayed at that pixel.
- The complexity depends upon the number of faces and the number of pixels to be considered.
- Mostly used
- Visibility is determined point by point at each pixel position on the projection plane.

For each pixel in the

image do Begin

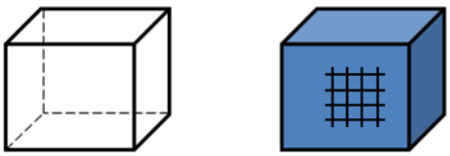
1. Determine the object closest to the viewer that is pierced by the projector through the pixel
2. Draw the pixel in the object color.

End

Two main types of algorithms:

– *Object space: Determine which part of the object are visible*

– *Image space: Determine per pixel which point of an object is visible*



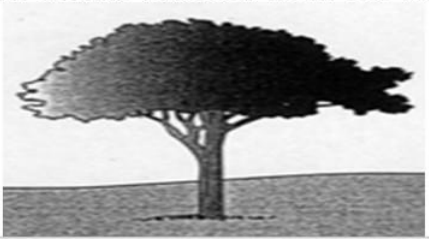
Object space Image space

<u>Visible Surface Detection Algorithm</u>

When we want to display a 3D object on a 2D screen, we need to identify those parts of a screen that are visible from a chosen viewing position.

Visible surface detection algorithms are based on following 2 concepts:

- **Sorting** : It facilitate the depth comparison by ordering the object surface or lines in the scene according to their distance from the view plane
- **Coherence** : This method are used to take advantage of regularities in a scene
  (It contain intervals of constant pixel intensity & scan pattern change (a little), from one line to other)



Z-BUFFER METHOD (Depth Buffer Method)

1. Depth-Buffer Method

A commonly used image-space approach to detecting visible surfaces is the depth-buffer method, which compares surface depths at each pixel position on the projection plane. This procedure is also referred to as the **Z-buffer method**, since object depth is usually measured from the view plane along the z axis of a viewing system. Each surface of a scene is processed separately, one point at a time across the surface. The method is usually applied to scenes containing only polygon surfaces, because depth values can be computed very quickly and the method is easy to implement. But the mcthod can be applied to nonplanar surfaces.

With object descriptions converted to projection coordinates, each *(x,* y, *z)* position on a polygon surface corresponds to the orthographic projection point *(x,* y) on the view plane. Therefore, for each pixel position *(x,* y) on the view plane, object depths can be compared by comparing z values.

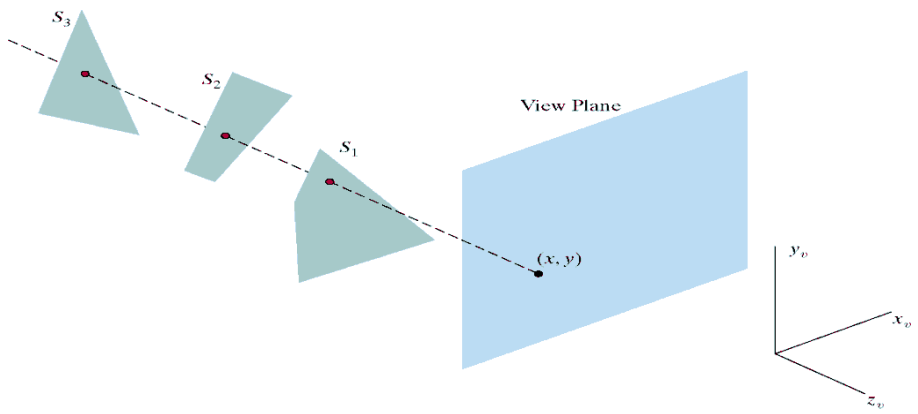


Figure shows three surfaces at varying distances along the orthographic projection line from **position (x,y) in a view plane .Surface** 1, is closest at this position, so its surface intensity value at **(x,** y) is saved. We can implement the depth-buffer algorithm in normalized coordinates,so that *z* values range from **0** at the back clipping plane to Z_{max} at the front clipping plane . The value of Z_{max} can be set either to 1 or to the largest value that can be stored on the system.

As implied by the name of this method, two buffer areas are required. A depth buffer is used to store depth values for each (x, y) position as surfaces are processed, and the refresh buffer stores the intensity values for each position. Initially,all positions in the depth buffer are set to 0 (minimum depth), and the refresh buffer is initialized to the background intensity. Each surface listed in the polygon tables is then processed, one scan line at a time, calculating the depth (z value) at each (x, y) pixel position. The calculated depth is compared to the value previously stored in the depth buffer at that position. If the calculated depth is greater than the value stored in the depth buffer, the new depth value is stored, and the surface intensity at that position is determined and in the same *xy* location in the refresh buffer.

Algorithm

1. Initialize the depth buffer and refresh buffer so that for all buffer posi-tions (x, y),

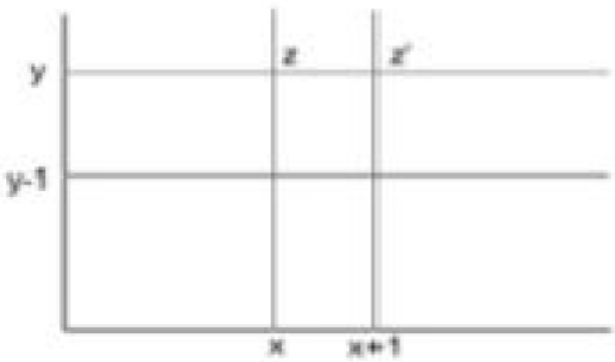
$$\text{depth}(x, y) = 0, \qquad \text{refresh}(x, y) = I_{\text{backgnd}}$$

2. For each position on each polygon surface, compare depth values to previously stored values in the depth buffer to determine visibility.
   - Calculate the depth z for each (x, y) position on the polygon.
   - If $z > \text{depth}(x, y)$, then set

$$\text{depth}(x, y) = z, \qquad \text{refresh}(x, y) = I_{\text{surf}}(x,y)$$

Depth values for a surface position (x, y) are calculated from the plane equation for each surface:

$$z = \frac{-Ax - By - D}{C}$$



For any scan line adjacent horizontal x positions or vertical y positions differ by 1 unit. The depth value of the next position (x+1,y) on the scan line can be obtained using

Surface Equation

AX+BY+CZ+D=0

Depth calculation along z axis

Z=(-AX-BY-D)/C

Z'=(-Ax - A- By - D)/C

Z'=(-Ax-By-D)/C – A/C

=Z- A/C

For Next point X+1

$$z' = \frac{-A(x+1) - By - D}{C}$$

$$= z - \frac{A}{C}$$

The ratio -*A/C* is constant for each surface, so succeeding depth values across a scan line are obtained from preceding values with a single addition.

Advantages

- It is easy to implement.
- It reduces the speed problem if implemented in hardware.

Disadvantages

- It requires large memory.
- It is time consuming process.
- Can find only one visible surface at each pixel position.
- Deals with only opaque surfaces.
- Cannot accumulate intensity values for more than one surface. (for transparent surfaces).

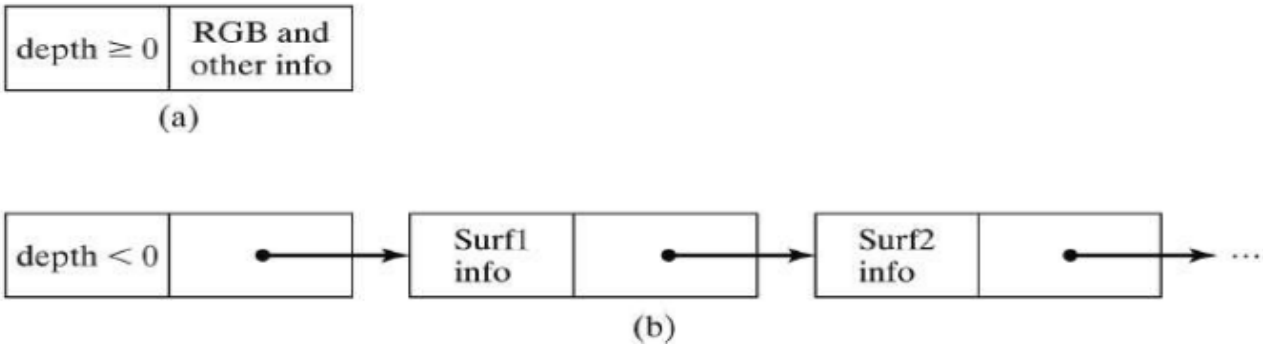
2. A-Buffer Method

- Z-buffer algorithm is not sufficient to work on surfaces which are transparent in nature.
- A buffer algorithm is used not only for transparent surface but also for opaque surface. Hence this algorithm is expansion of Z buffer algorithm.
- In a buffer algorithm a link list is attached with each pixel positions. This link list contains intensity information of each surface associated with it.

The A-buffer method is an extension of the depth-buffer method. The A-buffer expands on the depth buffer method to allow transparencies. The key data structure in the A-buffer is the accumulation buffer. Each position in the A-buffer has two fields −

- **Depth field** − It stores a positive or negative real number

- **Intensity field** − It stores surface-intensity information or a pointer value



If depth >= 0, the number stored at that position is the depth of a single surface overlapping the corresponding pixel area. The intensity field then stores the RGB components of the surface color at that point and the percent of pixel coverage.

If depth < 0, it indicates multiple-surface contributions to the pixel intensity. The intensity field then stores a pointer to a linked list of surface data. The surface buffer in the A-buffer includes

- RGB intensity components
- Opacity Parameter
- Depth
- Percent of area coverage
- Surface identifier

The algorithm proceeds just like the depth buffer algorithm. The depth and opacity values are used to determine the final color of a pixel.

3. Scanline Method

This image space method for removing hidden surface is an extension of the scan-line algorithm for polygon filling . As each scan line is processed, all polygon surfaces intersecting that line are examined to determine which are visible. Across each scan line, depth calculations are made for each overlapping surface to determine which is nearest to the view plane. When the visible surface has been determined, the intensity value for that position is entered into the refresh buffer.

Two important tables, edge table and polygon table, are maintained for this.

1. **The Edge Table** − It contains coordinate endpoints of each line in the scene, the inverse slope of each line, and pointers into the polygon table to connect edges to surfaces.

2. **The Polygon Table** − It contains the plane coefficients, surface material properties, other surface data, and may be pointers to the edge table.

- The scan-line algorithm is another image-space algorithm.
- It processes the image one scan-line at a time rather than one pixel at a time.
- By using area coherence of the polygon, the processing efficiency is improved over the pixel oriented method.

Scan-line hidden surface removal algorithm

1. Establish the necessary data structures.

a) Polygon table with coefficients, colour, and edge pointers.

b) Edge table with line end points, inverse slope, and polygon pointers.

c) Active edge list, sorted in order of increasing x.

d) An IN flag for each polygon.

2. Repeat for all scan lines:

a) Update active edge list by sorting edge table against scan line y value.

b) Scan across, using background colour, until an IN flag goes on.

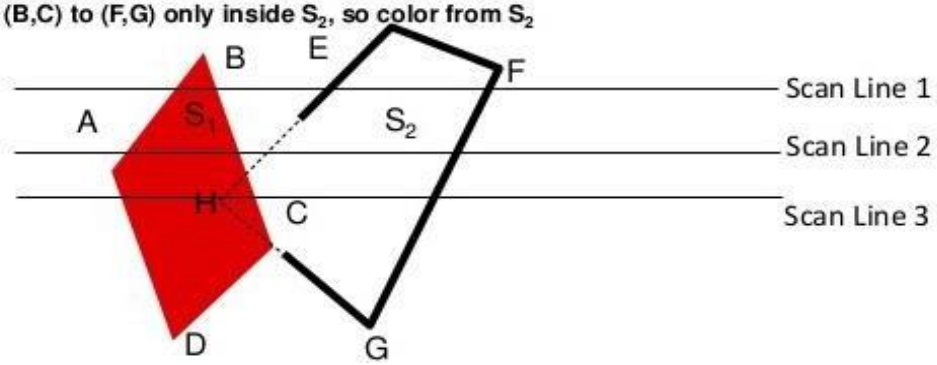
c) When 1 polygon flag is on for surface S , enter intensity (colour) into refresh buffer.

d) When 2 or more surface flags are on, do depth sort and use intensity In for surface n with maximum z-depth.

e) Use coherence of planes to repeat for next scan line

To facilitate the search for surfaces crossing a given scan-line, an active list of edges is formed. The active list stores only those edges that cross the scan-line in order of increasing x. Also a flag is set for each surface to indicate whether a position along a scan-line is either inside or outside the surface. Pixel positions across each scan-line are processed from left to right. At the left intersection with a surface, the surface flag is turned on and at the right, the flag is turned off. We only need to perform depth calculations when multiple surfaces have their flags turned on at a certain scan-line position.

- Scan Line 1:
  - (A,B) to (B,C) only inside S_1, so color from S_1 (Flag1 will on and 2 will off)
  - (E,H) to (F,G) only inside S_2, so color from S_2 (Flag 2 will on 1 will off)
- Scan Line 3:
  - (A,D) to (E,H) only inside S_1, so color from S_1
  - (E,H) to (B,C) inside S_1 and S_2, so compute & test depth. In this example we color from S_1
  - (B,C) to (F,G) only inside S_2, so color from S_2



The active list for scan line 1 contains information from the edge table for edges AB, BC, EH, and FG. For positions along this scan line between edges *AB* and BC, only the flag for surface S1 is on. Therefore no depth calculations are necessary, and intensity information for surface S1 is entered from the polygon table into the refresh buffer. Similarly, between edges EH and FG, only the flag for surface S2 is on.

For scan lines 2 and 3 ,the active edge list contains edges *AD,EH, BC,* and *FG.* Along scan line 2 from edge *AD* to edge *EH,* only the flag for surface S1 is on. But between edges EH and *BC,* the flags for both surfaces are on.In this interval, depth calculations must be made using the plane coefficients for the two surfaces. For this example, the depth of surface S1 is assumed to be less than that of S2, so intensities for surface S1 are loaded into the refresh buffer until boundary BC is encountered. Then the flag for surface S1 goes off, and intensities for surface *S2* are stored until edge FG is passed.

We can take advantage of-coherence along the scan lines as we pass from one scan line to the next. The scan line 3 has the same active list of edges as scan line 2. Since no changes have occurred in line intersections, it is unnecessary again to make depth calculations between edges *EH* and *BC.* The two surfaces must be in the same orientation as determined on scan line 2, so the intensities for surface S1 can be entered without further calculations.

<u>Advantage:</u> Any number of overlapping polygon surfaces can be processed with this scan-line method.